ZoneFood — Master Project Plan

Locality-Scale Food Resource & Waste Intelligence System — DBMS Level 3 Project, PES University

Consolidated edition

Contents

How to read this document **4**

PART A — CORE PLAN **5**

1. The problem, stated precisely . . . 5
2. Why three database paradigms (this is a graded justification — Req. 3, 18) . . . 5
   - 2.1 PostgreSQL 16 — the system of record . . . 6
   - 2.2 MongoDB — heterogeneous field logs . . . 6
   - 2.3 pgvector — semantic layer . . . 6
3. System architecture . . . 7
   - 3.1 Exact dependency list . . . 7
4. Database design — PostgreSQL . . . 7
   - 4.1 ER overview (entities and cardinalities) . . . 7
   - 4.2 Normalisation note (put this in the report — Req. 14) . . . 8
   - 4.3 Full DDL . . . 8
   - 4.4 Data dictionary (Req. 14) — how to produce it without typing it . . . 15
5. Advanced database features — Req. 7 needs four; here are nine . . . 15
   - 5.1 Indexing strategy (with a reason per index) . . . 15
   - 5.2 Views and materialised views . . . 16
   - 5.3 Stored procedures and functions . . . 17
   - 5.4 Triggers . . . 19
   - 5.5 Transactions . . . 20
   - 5.6 Full-text search . . . 21
   - 5.7 Vector similarity search . . . 21
   - 5.8 Window functions, CTEs, recursive CTE . . . 22
   - 5.9 Partitioning . . . 23
6. MongoDB design (Req. 4 — document the data model, indexes, retrieval) . . . 23
   - 6.1 Collections . . . 23
   - 6.2 Indexes and validation . . . 23
   - 6.3 Aggregation pipelines (Req. 7: “NoSQL aggregation”) . . . 24
   - 6.4 The ETL bridge (Mongo → Postgres) . . . 25
7. Query catalogue (Req. 6 — JOIN, GROUP BY, HAVING, ORDER BY, aggregates, subqueries) 25
8. Application design (Next.js — Req. 9, 10) . . . 27
   - 8.1 Route map . . . 27
   - 8.2 Data-access layer (one file, everything goes through it) . . . 27
   - 8.3 UI components worth building (Req. 9 asks for forms, tables, search, filtering, dashboards) . . . 28
9. Auth, roles, security (Req. 11, 12, 13) . . . 28
10. Seed data (do this in Week 2 — nothing works without it) . . . 28
11. Forecasting layer (keep it small — it is not what you’re graded on) . . . 29

12. Timeline and team split . . . 30
10-week plan . . . 30
13. Testing (Req. 14, 16 — “test cases”, “testing results”) . . . 31
14. Deliverables checklist (mapped to the requirements PDF) . . . 31
15. Demo script (12 minutes) . . . 32
16. Pitfalls that sink projects like this . . . 32
17. Immediate next steps . . . 33

PART B — TABLE CATALOGUE, PROCESSING PIPELINES & FRONTEND 34
B0. Table catalogue at a glance . . . 34
B1.1 GROUP A — Geography & Entities (6 tables) . . . 34
B1.2 GROUP B — Identity, Access & Audit (4 tables) . . . 36
B1.3 GROUP C — Food Reference / Master Data (6 tables) . . . 36
B1.4 GROUP D — Food Inflow (3 tables) . . . 37
B1.5 GROUP E — Preparation & Service (5 tables) . . . 38
B1.6 GROUP F — Waste (3 tables) . . . 39
B1.7 GROUP G — Redistribution (3 tables) . . . 40
B1.8 GROUP H — Context (2 tables) . . . 41
B1.9 GROUP I — Forecasting & Evaluation (4 tables) . . . 41
B1.10 GROUP J — Operational Intelligence (2 tables) . . . 42
B1.11 MongoDB collections (5) . . . 43
B1.12 Vector storage summary . . . 43
B1.13 Table count summary . . . 44
B2.0 The full pipeline at a glance . . . 44
B2.1 [P1] INGESTION — four write paths . . . 45
The structured write path in detail (`POST /api/procurement`) . . . 45
Where errors surface . . . 45
B2.2 [P2] NORMALISATION — the Mongo → Postgres ETL bridge . . . 45
Step-by-step . . . 45
Idempotency . . . 46
B2.3 [P3] DERIVATION — computation that lives in the database . . . 46
B2.4 [P4] FEATURE BUILD — turning the database into training data . . . 47
The 18 features, and the SQL that makes each . . . 47
Leakage guard . . . 48
Split . . . 48
B2.5 [P5] TRAINING . . . 48
Output of training . . . 49
B2.6 [P6] INFERENCE — how a prediction actually gets made . . . 49
Nightly batch (03:00) . . . 49
The recommendation logic, explained properly . . . 49
What `explanation` JSONB holds . . . 50
On-demand what-if . . . 50
B2.7 [P7] EVALUATION — closing the loop . . . 50
B2.8 [P8] DETECTION — five alerts, all pure SQL, zero ML . . . 51
B2.9 [P9] MATCHING — surplus to recipient . . . 51
B2.10 [P10] SCHEDULE SUMMARY . . . 51

PART 3: FRONTEND — WHO USES WHAT, AND WHY 52
B3.0 Five users, five completely different jobs . . . 52
B3.1 PERSONA 1 — Kitchen / household logger (`ESTAB_STAFF`) . . . 52
Screens . . . 53
The close-session screen in detail — the highest-value screen in the app . . . 53
Components . . . 53
What they cannot do . . . 54
B3.2 PERSONA 2 — Establishment manager (`ESTAB_MANAGER`) . . . 54
Screens . . . 54

The forecast screen is where the ML earns its place . . . 54
Peer comparison, handled carefully . . . 55
B3.3 PERSONA 3 — Recipient agent (RECIPIENT_AGENT) . . . 55
Screens . . . 55
The countdown is the design centrepiece . . . 56
B3.4 PERSONA 4 — Zone administrator (ZONE_ADMIN) . . . 56
Screens . . . 56
The unresolved-names queue is a small screen that impresses . . . 57
Charts worth building (Recharts) . . . 57
B3.5 PERSONA 5 — Auditor (AUDITOR) . . . 57
Screens . . . 57
B3.6 Shared frontend architecture . . . 58
Navigation . . . 58
Data fetching . . . 58
Reusable components . . . 58
Error surfacing . . . 58
Empty and loading states . . . 58
B3.7 Screen count and build order . . . 59
B3.8 The one-paragraph answer if an examiner asks “who is this for?” . . . 59

PART C — RESEARCH-GRADE EXTENSION (TIER T4) 60
From “locality food-waste database” to “food-resource intelligence system with a closed evaluation loop” . . . 60
C.1 The one-sentence reframe . . . 60
C.2 Four levels, and where each version of the project sits . . . 60
C.3 Where this sits in the literature (state this in your report’s related-work section) . . . 60
C.4 The central theory, formalised as a pipeline . . . 61
C.5 New table group K — Food State, Safety & Shelf Life . . . 62
Why K3 and K4 are two tables, not one . . . 62
DDL — db/migrations/010_food_state_and_safety.sql . . . 62
C.6 New table group L — Traceability, Intervention & Data Quality . . . 64
6.1 Why batch_lot_consumption is the single highest-value addition in this whole document 65
DDL — db/migrations/011_traceability_and_intervention.sql . . . 65
6.2 The ALTER + DROP migration you need anyway — make it earn its keep . . . 66
C.7 MongoDB extension — one new collection . . . 66
C.8 Updated tier system . . . 67
C.9 New processing pipelines . . . 68
[P11] STATE DERIVATION — nightly, pure SQL + one trigger . . . 68
[P12] INTERVENTION EVALUATION — the loop-closer . . . 68
[P13] DATA QUALITY SCORING — weekly, pure SQL . . . 68
C.10 New/extended query catalogue (add to your existing Req. 6 catalogue) . . . 69
C.11 Frontend additions (per persona, additive to your existing screen list) . . . 70
C.12 Five-layer evaluation framework (for the report’s “Evaluation” section) . . . 71
C.13 Revised phrasing for your model/forecasting narrative . . . 72
C.14 Scope guardrails — what NOT to add . . . 72
C.15 Where T4 fits in your existing 10-week plan . . . 73
C.16 One-paragraph project definition, for the abstract / viva opener . . . 73
C.17 Summary of every new object introduced in this document . . . 74

PART D — CONSOLIDATED REFERENCE 75
D1. Every storage object, every tier, one table . . . 75
D2. Requirements-to-deliverable map (unified) . . . 75
D3. The project, in one paragraph, at each depth . . . 76
D4. Closing checklist before you submit . . . 76

How to read this document

This is the single consolidated plan for ZoneFood. It replaces three earlier working documents by merging them into one, in the order you should actually build the project:

- **Part A — Core Plan.** The problem statement, the database-paradigm justification, system architecture, the full T1–T3 schema and DDL, advanced database features, MongoDB design, the query catalogue, application design, auth, seed data, the forecasting layer, timeline, testing, deliverables, demo script, and pitfalls. If you build only Part A, you have a complete, gradeable Level-3 DBMS project.
- **Part B — Table Catalogue, Processing Pipelines & Frontend.** A deeper reference layer: every table justified individually, every data-processing pipeline (ingestion through evaluation) explained step by step, and a persona-by-persona frontend design for the five roles in the system.
- **Part C — Research-Grade Extension (Tier T4).** An optional layer, built only after Parts A and B are solid and tested, that turns the project from "a very good database application" into something closer to a small research system: food *state* (not just quantity), safety/shelf-life modelling, traceability, intervention tracking, measured outcomes, and a stated position in the food-waste research literature. Every item in Part C is individually cuttable without damaging Parts A or B.
- **Part D — Consolidated Reference.** A single tier table (T1–T4) covering all ~49 storage objects across every part, a unified requirements-to-deliverable map, and the closing project statement to use in your abstract and viva.

If your timeline is tight: finish Part A end-to-end, use Part B as you build each piece, and treat all of Part C as stretch goals for the last available week. Section-by-section priority inside Part C is given in §C.8.

PART A — CORE PLAN

1. The problem, stated precisely

A defined urban zone contains heterogeneous food-handling entities — restaurants, colleges, hostels, offices, households, clubs, supermarkets, hospitals. Each has food **inflow** (procurement, preparation) and **outflow** (consumption, redistribution, waste). Today these are informational islands: nobody can answer zone-level questions.

The system answers six question classes. Every one of them is a database question, and every one maps to specific schema objects later in this document:

#	Question	Why it needs a database, not a spreadsheet
Q1	**Where** is food wasted, and at what *stage*?	Requires joining inflow → preparation → service → waste across entity types with different units and different logging schemas.
Q2	**Why** is it wasted?	Free-text reasons must be normalised into comparable causes → vector similarity + lookup tables.
Q3	**Which food is still edible and about to be lost?**	Time-windowed queries over lot expiry + consumption velocity; drives an alerting trigger.
Q4	**Can surplus at A be matched to need at B, in time?**	Semantic matching (dietary text, quantity, pickup window) + a concurrency-safe claim transaction.
Q5	**Who is an outlier vs. their peers?**	Window functions over normalised waste intensity, benchmarked within establishment category.
Q6	**What will demand be, and did we predict it well?**	Forecast rows stored in SQL; error computed by SQL, not by the model.

The unit of insight is **not** "kg wasted". It is **waste intensity** — kg per served meal, per occupant, per ₹1000 of procurement — because that is the only way a 500-household block and a 40-seat café can be compared. This single decision is what makes the analytics defensible in a viva.

2. Why three database paradigms (this is a graded justification — Req. 3, 18)

Do not write "we used MongoDB to show we can use MongoDB." Write this instead:

2.1 PostgreSQL 16 — the system of record

Holds: entities, the food lifecycle chain, quantities, money, claims, forecasts.

Justification: every row in the lifecycle is a quantity that must balance. `procured ≥ prepared`, `served + leftover ≤ prepared`, `claimed_qty ≤ available_qty`. These are *invariants*, not preferences — they need declarative CHECK constraints, foreign keys and ACID transactions. The surplus-claim flow has two NGOs racing for the same 12 kg of biryani; without row-level locking you double-allocate food that doesn't exist. That is a textbook case for a relational engine and nothing else.

Why PostgreSQL over MySQL: pgvector, materialised views, native partitioning, `FILTER` clauses, richer window functions, and — decisively — it lets you host the vector paradigm inside the same engine instead of running a fourth service. Your draft's MySQL choice was inherited from the old brief; there is no reason to keep it.

2.2 MongoDB — heterogeneous field logs

Holds: raw daily waste submissions, inspection reports, flexible establishment metadata, smart-bin readings, activity stream.

Justification (the important one): *different establishment types cannot log the same shape of data.*

- A restaurant logs per-dish plate waste: `[{dish, portions_returned, avg_plate_leftover_g}, ...]`
- A household logs bin categories: `{peels_kg, cooked_leftovers_kg, expired_packaged_kg, photo_url}`
- A college mess logs per-counter counts and headcounts by meal slot
- A supermarket logs SKU-level markdown/expiry with barcode arrays

Forcing these into one relational table produces either 40 nullable columns or an EAV anti-pattern. Both are worse designs. Mongo stores each submission in its natural shape; a nightly ETL projects the *common, validated subset* into Postgres `waste_event`. Raw stays in Mongo (audit trail, re-processable), normalised lands in SQL (analytics, constraints).

This is the honest architectural pattern — **document store at the edge, relational at the core** — and it's exactly the kind of reasoning Req. 18 asks for.

2.3 pgvector — semantic layer

Holds: embeddings of surplus descriptions, recipient need profiles, waste-cause notes, food name variants.

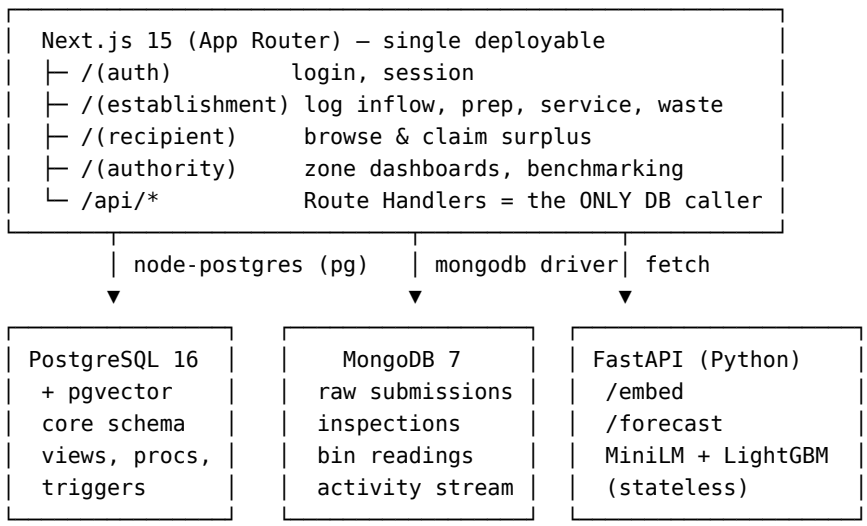
Justification: three real problems that keyword search cannot solve.

1. **Surplus ↔ need matching.** Listing: *"12 kg cooked veg pulao, Jain (no onion/garlic), ready 8:30pm, collect by 10pm."* Need: *"we serve ~200 vegetarian dinners nightly, van available after 7pm."* No shared keywords, high semantic overlap. Cosine similarity over sentence embeddings solves it; `LIKE '%veg%'` does not.
2. **Food-name normalisation.** Staff type `Veg Biriyani`, `vegetable biryani`, `Veg Biriyani (Special)`. Nearest-neighbour against canonical `food_item` embeddings collapses them, which is what makes Q1's "top wasted items" ranking trustworthy at all.
3. **Root-cause clustering.** Free-text `waste_event.notes` → nearest neighbours → "show me the 20 most similar past incidents" → recurring causes surface without a taxonomy anyone had to design upfront.

Model: `sentence-transformers/all-MiniLM-L6-v2`, 384-dim, runs locally, free. Metric: cosine. Index: HNSW (`vector_cosine_ops`).

> Fallback if pgvector install fights you: MongoDB Atlas Vector Search is an acceptable substitute and still counts. But local pgvector via Docker (`pgvector/pgvector:pg16`) is one line and almost never fails.

3. System architecture



Rules that keep this clean: - The browser never touches a database. All access is through Next.js Route Handlers (Req. 10). - The Python service is **stateless** — it embeds and predicts, it never owns data. Everything it produces is written back to Postgres by Node. This keeps the DB the system of record, which is the point of the course. - All credentials in `.env.local` / `.env`, never in source, `.env*` in `.gitignore`, `.env.example` committed (Req. 10, 13).

3.1 Exact dependency list

Next.js app

```
next@15 react react-dom typescript
pg @types/pg                    # raw parameterised SQL — faculty can read it
mongodb                         # official driver
jose bcryptjs zod               # JWT in httpOnly cookie, hashing, input validation
recharts date-fns
tailwindcss @tailwindcss/postcss
```

Python service

```
fastapi uvicorn pydantic
sentence-transformers torch     # embeddings
pandas numpy scikit-learn lightgbm joblib
psycopg[binary]                 # only for the batch backfill script
```

Infra

```
docker compose: pgvector/pgvector:pg16 , mongo:7
```

> **On ORMs:** use raw SQL through `pg` with parameterised queries (`$1, $2`). Prisma/Drizzle would hide exactly the DDL and query craft you are being graded on, and Prisma actively fights views, triggers and vector columns. Keep every `CREATE` statement in numbered files under `db/migrations/`. That folder *is* your Req. 5 evidence.

4. Database design — PostgreSQL

4.1 ER overview (entities and cardinalities)

```
zone ──self-ref(parent_zone_id)──┐
  │1                             │
```

```
  │N
establishment ──1:1──> commercial_profile | institutional_profile | residential_profile
  │1                    (disjoint specialisation, PK = FK)
  ├──N app_user
  ├──N procurement ──1:N── procurement_line ──N:1── food_item ──N:1── food_category
  │                              │1
  │                              └──1:N inventory_lot ──N:1── food_item
  ├──N dish ──M:N(dish_ingredient)── food_item
  ├──N service_session ──1:N── preparation_batch ──N:1── dish
  │                                    │1
  │                                    └──1:1 consumption_record
  ├──N waste_event ──N:1── waste_reason , disposal_route , food_item
  ├──N surplus_listing ──1:N── redistribution_claim ──N:1── recipient_org
  └──N demand_forecast

locality_event ──M:N(zone_event)── zone
emission_factor ──1:N── food_category
audit_log (written by triggers, references app_user)
```

Three modelling decisions worth defending in the viva:

1. **Specialisation on `establishment`.** A household and a hotel are both establishments (same lifecycle, same waste events) but carry different attributes. Supertype + three disjoint subtype tables keeps `establishment` in 3NF instead of filling it with nulls, and lets `waste_event` have exactly one FK target.
2. **`zone` is self-referencing.** Sector → block → street lets you aggregate at any depth with one recursive CTE, and gives you a legitimate hierarchical query to demo.
3. **`waste_event` is the fact table.** Everything else is dimension or lineage. Designing it as a narrow, high-volume, append-mostly table is what makes partitioning and index strategy meaningful rather than decorative.

4.2 Normalisation note (put this in the report — Req. 14)

State the progression explicitly for at least two tables. Worked example for `waste_event`:

- **Unnormalised source:** the Mongo submission `{establishment_name, address, zone, items:[{food_name, category, kg, reason_text, disposal}]}`
- **1NF:** flatten the `items` array — one row per item; no repeating groups.
- **2NF:** `establishment_name`, `address`, `zone` depend only on the establishment, not on the (establishment, item, date) key → move to `establishment`.
- **3NF:** `food_category` and `emission_factor` are transitively dependent on `food_id`, not on the waste event → move to `food_item` / `food_category` / `emission_factor`.
- Result: `waste_event` holds only facts about the waste occurrence itself, plus FKs.

Note honestly where you **denormalise deliberately**: `waste_event.cost_estimate` is derived (`quantity_kg × unit_cost`) but stored, because unit cost drifts over time and a report run in March must reproduce March's numbers. That is a defensible, intentional violation — examiners like seeing that you know the difference.

4.3 Full DDL

Split across migration files as shown. This is the real schema; type it as-is.

`db/migrations/001_extensions_and_enums.sql`

```sql
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
```

```sql
CREATE TYPE establishment_kind AS ENUM
  ('COMMERCIAL','EDUCATIONAL','ORGANIZATIONAL','RESIDENTIAL','RECREATIONAL','RETAIL_FOOD','HEALTHCARE');
CREATE TYPE user_role AS ENUM
  ('ZONE_ADMIN','ESTAB_MANAGER','ESTAB_STAFF','RECIPIENT_AGENT','AUDITOR');
CREATE TYPE meal_slot AS ENUM ('BREAKFAST','LUNCH','SNACKS','DINNER','ALL_DAY');
CREATE TYPE waste_stage AS ENUM
  ('PRE_CONSUMER_PREP','PRE_CONSUMER_SPOILAGE','PRE_CONSUMER_EXPIRY','POST_CONSUMER_PLATE','POST_CONSUMER_SERVIC
CREATE TYPE listing_status AS ENUM ('OPEN','PARTIALLY_CLAIMED','FULLY_CLAIMED','EXPIRED','CANCELLED');
CREATE TYPE claim_status  AS ENUM ('REQUESTED','APPROVED','COLLECTED','NO_SHOW','REJECTED');
```

db/migrations/002_geography_and_entities.sql

```sql
CREATE TABLE zone (
  zone_id        SERIAL PRIMARY KEY,
  parent_zone_id INT REFERENCES zone(zone_id) ON DELETE RESTRICT,
  zone_name      VARCHAR(120) NOT NULL,
  zone_level     SMALLINT NOT NULL CHECK (zone_level BETWEEN 1 AND 4), -- 1=sector 4=street
  pincode        CHAR(6),
  centroid_lat   NUMERIC(9,6),
  centroid_lng   NUMERIC(9,6),
  CONSTRAINT uq_zone_name_parent UNIQUE (zone_name, parent_zone_id),
  CONSTRAINT ck_zone_not_self CHECK (zone_id <> parent_zone_id)
);

CREATE TABLE establishment_category (
  category_id    SERIAL PRIMARY KEY,
  kind           establishment_kind NOT NULL,
  category_name  VARCHAR(80) NOT NULL UNIQUE,   -- 'Fine Dining', 'College Mess', 'Apartment Block'
  capacity_metric VARCHAR(30) NOT NULL          -- 'seats' | 'students' | 'residents'
);

CREATE TABLE establishment (
  establishment_id  SERIAL PRIMARY KEY,
  zone_id           INT NOT NULL REFERENCES zone(zone_id) ON DELETE RESTRICT,
  category_id       INT NOT NULL REFERENCES establishment_category(category_id),
  name              VARCHAR(150) NOT NULL,
  address_line      VARCHAR(255),
  lat               NUMERIC(9,6),
  lng               NUMERIC(9,6),
  capacity          INT NOT NULL CHECK (capacity > 0),
  operational_since DATE NOT NULL,
  is_active         BOOLEAN NOT NULL DEFAULT TRUE,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_estab_name_zone UNIQUE (name, zone_id),
  CONSTRAINT ck_estab_since CHECK (operational_since <= CURRENT_DATE)
);

-- Disjoint specialisation: PK is also the FK
CREATE TABLE commercial_profile (
  establishment_id INT PRIMARY KEY REFERENCES establishment(establishment_id) ON DELETE CASCADE,
  cuisine_type     VARCHAR(60),
  seating_capacity INT CHECK (seating_capacity >= 0),
  avg_daily_covers INT CHECK (avg_daily_covers >= 0),
  has_buffet       BOOLEAN NOT NULL DEFAULT FALSE,
  fssai_license    VARCHAR(20) UNIQUE
);
```

```sql
CREATE TABLE institutional_profile (
  establishment_id  INT PRIMARY KEY REFERENCES establishment(establishment_id) ON DELETE CASCADE,
  enrolled_strength INT NOT NULL CHECK (enrolled_strength > 0),
  mess_type         VARCHAR(30) CHECK (mess_type IN ('VEG','NON_VEG','MIXED')),
  serves_meals_per_day SMALLINT NOT NULL CHECK (serves_meals_per_day BETWEEN 1 AND 5),
  academic_calendar_ref VARCHAR(60)
);

CREATE TABLE residential_profile (
  establishment_id INT PRIMARY KEY REFERENCES establishment(establishment_id) ON DELETE CASCADE,
  household_count  INT NOT NULL CHECK (household_count > 0),
  total_occupants  INT NOT NULL CHECK (total_occupants > 0),
  has_common_kitchen BOOLEAN NOT NULL DEFAULT FALSE,
  segregation_compliant BOOLEAN NOT NULL DEFAULT FALSE,
  CONSTRAINT ck_occupants_ge_households CHECK (total_occupants >= household_count)
);
```

db/migrations/003_users_auth.sql

```sql
CREATE TABLE app_user (
  user_id          SERIAL PRIMARY KEY,
  establishment_id INT REFERENCES establishment(establishment_id) ON DELETE SET NULL,
  recipient_id     INT,  -- FK added after recipient_org exists
  email            VARCHAR(150) NOT NULL UNIQUE,
  password_hash    VARCHAR(255) NOT NULL,      -- bcrypt, cost 12. NEVER plaintext (Req.13)
  full_name        VARCHAR(120) NOT NULL,
  role             user_role NOT NULL,
  phone            VARCHAR(15) CHECK (phone ~ '^[0-9]{10}$'),
  is_active        BOOLEAN NOT NULL DEFAULT TRUE,
  last_login_at    TIMESTAMPTZ,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  -- role/tenant coherence enforced in the database, not just in JS
  CONSTRAINT ck_user_scope CHECK (
    (role IN ('ESTAB_MANAGER','ESTAB_STAFF') AND establishment_id IS NOT NULL AND recipient_id IS NULL)
 OR (role = 'RECIPIENT_AGENT' AND recipient_id IS NOT NULL AND establishment_id IS NULL)
 OR (role IN ('ZONE_ADMIN','AUDITOR') AND establishment_id IS NULL AND recipient_id IS NULL)
  )
);
```

db/migrations/004_food_catalog_and_inflow.sql

```sql
CREATE TABLE food_category (
  food_category_id SERIAL PRIMARY KEY,
  category_name    VARCHAR(60) NOT NULL UNIQUE,  -- Cereals, Cooked Rice Dishes, Dairy, Produce...
  is_perishable    BOOLEAN NOT NULL DEFAULT TRUE,
  default_shelf_life_hours INT NOT NULL CHECK (default_shelf_life_hours > 0)
);

CREATE TABLE emission_factor (
  food_category_id INT PRIMARY KEY REFERENCES food_category(food_category_id),
  kg_co2e_per_kg   NUMERIC(6,3) NOT NULL CHECK (kg_co2e_per_kg >= 0),
  litres_water_per_kg NUMERIC(8,2) CHECK (litres_water_per_kg >= 0),
  source_citation  VARCHAR(200)
);

CREATE TABLE food_item (
```

```
  food_id          SERIAL PRIMARY KEY,
  food_category_id INT NOT NULL REFERENCES food_category(food_category_id),
  canonical_name   VARCHAR(120) NOT NULL UNIQUE,
  description      TEXT,
  base_unit        VARCHAR(10) NOT NULL DEFAULT 'kg' CHECK (base_unit IN ('kg','litre','unit')),
  avg_cost_per_unit NUMERIC(10,2) NOT NULL CHECK (avg_cost_per_unit >= 0),
  is_edible_as_is  BOOLEAN NOT NULL DEFAULT TRUE,
  name_embedding   vector(384),                -- pgvector: name normalisation
  search_doc       tsvector GENERATED ALWAYS AS
                     (to_tsvector('english', canonical_name || ' ' || COALESCE(description,''))) STORED
);

CREATE TABLE supplier (
  supplier_id  SERIAL PRIMARY KEY,
  name         VARCHAR(150) NOT NULL,
  zone_id      INT REFERENCES zone(zone_id),
  contact_phone VARCHAR(15),
  supplier_type VARCHAR(40) CHECK (supplier_type IN ('WHOLESALE','FARM','DISTRIBUTOR','RETAIL','OTHER'))
);

CREATE TABLE procurement (
  procurement_id   BIGSERIAL PRIMARY KEY,
  establishment_id INT NOT NULL REFERENCES establishment(establishment_id) ON DELETE CASCADE,
  supplier_id      INT REFERENCES supplier(supplier_id) ON DELETE SET NULL,
  invoice_no       VARCHAR(50),
  received_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  total_cost       NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (total_cost >= 0),
  recorded_by      INT REFERENCES app_user(user_id),
  CONSTRAINT uq_invoice UNIQUE (establishment_id, supplier_id, invoice_no)
);

CREATE TABLE procurement_line (
  line_id        BIGSERIAL PRIMARY KEY,
  procurement_id BIGINT NOT NULL REFERENCES procurement(procurement_id) ON DELETE CASCADE,
  food_id        INT NOT NULL REFERENCES food_item(food_id),
  quantity       NUMERIC(10,3) NOT NULL CHECK (quantity > 0),
  unit_cost      NUMERIC(10,2) NOT NULL CHECK (unit_cost >= 0),
  expiry_date    DATE,
  CONSTRAINT uq_proc_line UNIQUE (procurement_id, food_id)
);

CREATE TABLE inventory_lot (
  lot_id         BIGSERIAL PRIMARY KEY,
  establishment_id INT NOT NULL REFERENCES establishment(establishment_id) ON DELETE CASCADE,
  food_id        INT NOT NULL REFERENCES food_item(food_id),
  source_line_id BIGINT REFERENCES procurement_line(line_id) ON DELETE SET NULL,
  received_qty   NUMERIC(10,3) NOT NULL CHECK (received_qty > 0),
  remaining_qty  NUMERIC(10,3) NOT NULL CHECK (remaining_qty >= 0),
  unit_cost      NUMERIC(10,2) NOT NULL CHECK (unit_cost >= 0),
  received_at    TIMESTAMPTZ NOT NULL,
  expires_at     TIMESTAMPTZ NOT NULL,
  CONSTRAINT ck_lot_balance CHECK (remaining_qty <= received_qty),
  CONSTRAINT ck_lot_dates   CHECK (expires_at > received_at)
);
```

db/migrations/005_preparation_and_service.sql

```
CREATE TABLE dish (
  dish_id          SERIAL PRIMARY KEY,
  establishment_id INT NOT NULL REFERENCES establishment(establishment_id) ON DELETE CASCADE,
  dish_name        VARCHAR(120) NOT NULL,
  standard_portion_g NUMERIC(7,2) NOT NULL CHECK (standard_portion_g > 0),
  is_vegetarian    BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT uq_dish UNIQUE (establishment_id, dish_name)
);

CREATE TABLE dish_ingredient (            -- resolves M:N dish <-> food_item
  dish_id   INT NOT NULL REFERENCES dish(dish_id) ON DELETE CASCADE,
  food_id   INT NOT NULL REFERENCES food_item(food_id) ON DELETE RESTRICT,
  qty_per_portion NUMERIC(8,4) NOT NULL CHECK (qty_per_portion > 0),
  PRIMARY KEY (dish_id, food_id)
);

CREATE TABLE service_session (
  session_id       BIGSERIAL PRIMARY KEY,
  establishment_id INT NOT NULL REFERENCES establishment(establishment_id) ON DELETE CASCADE,
  service_date     DATE NOT NULL,
  slot             meal_slot NOT NULL,
  expected_headcount INT NOT NULL CHECK (expected_headcount >= 0),
  actual_headcount   INT CHECK (actual_headcount >= 0),
  is_closed        BOOLEAN NOT NULL DEFAULT FALSE,
  closed_at        TIMESTAMPTZ,
  CONSTRAINT uq_session UNIQUE (establishment_id, service_date, slot),
  CONSTRAINT ck_closed_has_actuals CHECK (NOT is_closed OR actual_headcount IS NOT NULL)
);

CREATE TABLE preparation_batch (
  batch_id      BIGSERIAL PRIMARY KEY,
  session_id    BIGINT NOT NULL REFERENCES service_session(session_id) ON DELETE CASCADE,
  dish_id       INT NOT NULL REFERENCES dish(dish_id),
  prepared_qty_kg  NUMERIC(10,3) NOT NULL CHECK (prepared_qty_kg > 0),
  prepared_portions INT NOT NULL CHECK (prepared_portions > 0),
  prep_cost     NUMERIC(10,2) CHECK (prep_cost >= 0),
  prepared_at   TIMESTAMPTZ NOT NULL,
  CONSTRAINT uq_batch UNIQUE (session_id, dish_id)
);

CREATE TABLE consumption_record (
  batch_id       BIGINT PRIMARY KEY REFERENCES preparation_batch(batch_id) ON DELETE CASCADE,
  served_qty_kg  NUMERIC(10,3) NOT NULL CHECK (served_qty_kg >= 0),
  leftover_qty_kg NUMERIC(10,3) NOT NULL CHECK (leftover_qty_kg >= 0),
  plate_waste_kg NUMERIC(10,3) NOT NULL DEFAULT 0 CHECK (plate_waste_kg >= 0),
  recorded_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  recorded_by    INT REFERENCES app_user(user_id)
  -- served + leftover <= prepared is cross-table: enforced by trigger (see 5.4)
);
```

db/migrations/006_waste_and_redistribution.sql

```
CREATE TABLE waste_reason (
  reason_id   SERIAL PRIMARY KEY,
  reason_code VARCHAR(40) NOT NULL UNIQUE,      -- OVER_PREPARATION, LOW_TURNOUT, SPOILAGE...
```

```
  description VARCHAR(200) NOT NULL,
  is_preventable BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE disposal_route (
  route_id    SERIAL PRIMARY KEY,
  route_name  VARCHAR(40) NOT NULL UNIQUE,      -- LANDFILL, COMPOST, BIOGAS, ANIMAL_FEED, DONATION
  is_diversion BOOLEAN NOT NULL,                -- anything but landfill = diverted
  cost_per_kg NUMERIC(8,2) NOT NULL DEFAULT 0 CHECK (cost_per_kg >= 0)
);

CREATE TABLE waste_event (
  waste_id         BIGSERIAL,
  establishment_id INT NOT NULL REFERENCES establishment(establishment_id) ON DELETE CASCADE,
  food_id          INT REFERENCES food_item(food_id),
  session_id       BIGINT REFERENCES service_session(session_id) ON DELETE SET NULL,
  lot_id           BIGINT REFERENCES inventory_lot(lot_id) ON DELETE SET NULL,
  reason_id        INT NOT NULL REFERENCES waste_reason(reason_id),
  route_id         INT NOT NULL REFERENCES disposal_route(route_id),
  stage            waste_stage NOT NULL,
  quantity_kg      NUMERIC(10,3) NOT NULL CHECK (quantity_kg > 0),
  was_edible       BOOLEAN NOT NULL DEFAULT TRUE,
  cost_estimate    NUMERIC(10,2) CHECK (cost_estimate >= 0),   -- deliberate denormalisation
  co2e_estimate    NUMERIC(10,3) CHECK (co2e_estimate >= 0),
  notes            TEXT,
  notes_embedding  vector(384),                                -- pgvector: cause clustering
  occurred_on      DATE NOT NULL,
  recorded_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  source_doc_id    VARCHAR(30),                                -- Mongo _id of the raw submission
  PRIMARY KEY (waste_id, occurred_on),
  CONSTRAINT ck_waste_not_future CHECK (occurred_on <= CURRENT_DATE)
) PARTITION BY RANGE (occurred_on);

-- create 12–24 monthly partitions; script this in a DO block
CREATE TABLE waste_event_2025_09 PARTITION OF waste_event
  FOR VALUES FROM ('2025-09-01') TO ('2025-10-01');
-- ... repeat per month ...
CREATE TABLE waste_event_default PARTITION OF waste_event DEFAULT;

CREATE TABLE recipient_org (
  recipient_id   SERIAL PRIMARY KEY,
  zone_id        INT NOT NULL REFERENCES zone(zone_id),
  org_name       VARCHAR(150) NOT NULL UNIQUE,
  org_type       VARCHAR(40) CHECK (org_type IN ('NGO','SHELTER','COMMUNITY_FRIDGE','ANIMAL_SHELTER','COMPOST_UN
  need_profile   TEXT NOT NULL,                 -- free text: what they need, when, capacity
  need_embedding vector(384),                   -- pgvector: matching
  daily_capacity_kg NUMERIC(8,2) NOT NULL CHECK (daily_capacity_kg > 0),
  accepts_cooked BOOLEAN NOT NULL DEFAULT TRUE,
  pickup_window_start TIME, pickup_window_end TIME,
  contact_phone  VARCHAR(15),
  CONSTRAINT ck_pickup_window CHECK (pickup_window_end > pickup_window_start)
);

ALTER TABLE app_user
  ADD CONSTRAINT fk_user_recipient FOREIGN KEY (recipient_id)
```

```
  REFERENCES recipient_org(recipient_id) ON DELETE SET NULL;

CREATE TABLE surplus_listing (
  listing_id       BIGSERIAL PRIMARY KEY,
  establishment_id INT NOT NULL REFERENCES establishment(establishment_id) ON DELETE CASCADE,
  food_id          INT REFERENCES food_item(food_id),
  session_id       BIGINT REFERENCES service_session(session_id) ON DELETE SET NULL,
  description      TEXT NOT NULL,
  desc_embedding   vector(384),                -- pgvector: matching
  offered_qty_kg   NUMERIC(10,3) NOT NULL CHECK (offered_qty_kg > 0),
  claimed_qty_kg   NUMERIC(10,3) NOT NULL DEFAULT 0 CHECK (claimed_qty_kg >= 0),
  is_cooked        BOOLEAN NOT NULL DEFAULT TRUE,
  is_vegetarian    BOOLEAN NOT NULL DEFAULT TRUE,
  available_from   TIMESTAMPTZ NOT NULL,
  expires_at       TIMESTAMPTZ NOT NULL,
  status           listing_status NOT NULL DEFAULT 'OPEN',
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT ck_listing_balance CHECK (claimed_qty_kg <= offered_qty_kg),
  CONSTRAINT ck_listing_window  CHECK (expires_at > available_from)
);

CREATE TABLE redistribution_claim (
  claim_id     BIGSERIAL PRIMARY KEY,
  listing_id   BIGINT NOT NULL REFERENCES surplus_listing(listing_id) ON DELETE CASCADE,
  recipient_id INT NOT NULL REFERENCES recipient_org(recipient_id) ON DELETE CASCADE,
  claimed_qty_kg NUMERIC(10,3) NOT NULL CHECK (claimed_qty_kg > 0),
  match_score  NUMERIC(5,4) CHECK (match_score BETWEEN 0 AND 1),  -- cosine similarity at match time
  status       claim_status NOT NULL DEFAULT 'REQUESTED',
  requested_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  collected_at TIMESTAMPTZ,
  CONSTRAINT uq_claim UNIQUE (listing_id, recipient_id),
  CONSTRAINT ck_collected CHECK (status <> 'COLLECTED' OR collected_at IS NOT NULL)
);
```

db/migrations/007_events_forecasting_audit.sql

```
CREATE TABLE locality_event (
  event_id    SERIAL PRIMARY KEY,
  event_name  VARCHAR(120) NOT NULL,
  event_type  VARCHAR(40) CHECK (event_type IN ('FESTIVAL','PUBLIC_HOLIDAY','EXAM_PERIOD','VACATION','LOCAL_EVEN
  start_date  DATE NOT NULL,
  end_date    DATE NOT NULL,
  expected_impact SMALLINT CHECK (expected_impact BETWEEN -2 AND 2), -- demand shift
  CONSTRAINT ck_event_dates CHECK (end_date >= start_date)
);

CREATE TABLE zone_event (                  -- M:N zone <-> locality_event
  zone_id  INT NOT NULL REFERENCES zone(zone_id) ON DELETE CASCADE,
  event_id INT NOT NULL REFERENCES locality_event(event_id) ON DELETE CASCADE,
  PRIMARY KEY (zone_id, event_id)
);

CREATE TABLE forecast_model (
  model_version VARCHAR(30) PRIMARY KEY,        -- 'lgbm_v3'
  algorithm     VARCHAR(40) NOT NULL,
  trained_at    TIMESTAMPTZ NOT NULL,
```

```
  feature_list  TEXT NOT NULL,
  train_rows    INT CHECK (train_rows > 0),
  is_active     BOOLEAN NOT NULL DEFAULT FALSE
);
CREATE UNIQUE INDEX uq_one_active_model ON forecast_model (is_active) WHERE is_active;

CREATE TABLE demand_forecast (
  forecast_id      BIGSERIAL PRIMARY KEY,
  establishment_id INT NOT NULL REFERENCES establishment(establishment_id) ON DELETE CASCADE,
  dish_id          INT REFERENCES dish(dish_id) ON DELETE SET NULL,
  target_date      DATE NOT NULL,
  slot             meal_slot NOT NULL,
  model_version    VARCHAR(30) NOT NULL REFERENCES forecast_model(model_version),
  predicted_qty_kg NUMERIC(10,3) NOT NULL CHECK (predicted_qty_kg >= 0),
  recommended_qty_kg NUMERIC(10,3) NOT NULL CHECK (recommended_qty_kg >= 0),
  safety_buffer_pct NUMERIC(5,2) NOT NULL CHECK (safety_buffer_pct BETWEEN 0 AND 100),
  explanation      JSONB,                       -- drivers shown in the UI
  generated_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_forecast UNIQUE (establishment_id, dish_id, target_date, slot, model_version)
);

CREATE TABLE audit_log (
  audit_id    BIGSERIAL PRIMARY KEY,
  table_name  VARCHAR(60) NOT NULL,
  record_pk   VARCHAR(60) NOT NULL,
  action      CHAR(1) NOT NULL CHECK (action IN ('I','U','D')),
  changed_by  INT REFERENCES app_user(user_id),
  old_row     JSONB,
  new_row     JSONB,
  changed_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

4.4 Data dictionary (Req. 14) — how to produce it without typing it

Don't hand-write it. Generate it from the catalog and paste into the report:

```
SELECT c.table_name, c.column_name, c.data_type,
       c.is_nullable, c.column_default,
       pg_catalog.col_description(fc.oid, c.ordinal_position) AS comment
FROM information_schema.columns c
JOIN pg_class fc ON fc.relname = c.table_name
WHERE c.table_schema = 'public'
ORDER BY c.table_name, c.ordinal_position;
```

Add `COMMENT ON COLUMN waste_event.stage IS '...';` statements in a `008_comments.sql` migration so the comment column fills itself.

5. Advanced database features — Req. 7 needs four; here are nine

Implement all of these. Each is a slide in your demo.

5.1 Indexing strategy (with a reason per index)

```
-- Fact table: every dashboard filters by establishment + date range
CREATE INDEX idx_waste_estab_date ON waste_event (establishment_id, occurred_on DESC);
```

```sql
-- Ranking "top wasted foods in zone" scans by food across all establishments
CREATE INDEX idx_waste_food_date  ON waste_event (food_id, occurred_on);
-- Stage breakdown is a low-cardinality filter -> include it in a covering index
CREATE INDEX idx_waste_stage_cover ON waste_event (occurred_on, stage) INCLUDE (quantity_kg, establishment_id);
-- PARTIAL index: the marketplace only ever queries OPEN listings. ~5% of rows.
CREATE INDEX idx_listing_open ON surplus_listing (expires_at)
  WHERE status IN ('OPEN','PARTIALLY_CLAIMED');
-- Expiry sweep runs every hour over a tiny slice
CREATE INDEX idx_lot_expiry ON inventory_lot (establishment_id, expires_at)
  WHERE remaining_qty > 0;
-- Full-text
CREATE INDEX idx_food_fts ON food_item USING GIN (search_doc);
-- Fuzzy fallback for typo'd food names
CREATE INDEX idx_food_trgm ON food_item USING GIN (canonical_name gin_trgm_ops);
-- Vector (HNSW: fast recall, build once)
CREATE INDEX idx_listing_vec ON surplus_listing USING hnsw (desc_embedding vector_cosine_ops);
CREATE INDEX idx_recipient_vec ON recipient_org USING hnsw (need_embedding vector_cosine_ops);
CREATE INDEX idx_wastenote_vec ON waste_event USING hnsw (notes_embedding vector_cosine_ops);
-- Time-ordered append-only table: BRIN is ~1000x smaller than B-tree here
CREATE INDEX idx_audit_brin ON audit_log USING BRIN (changed_at);
```

Demo this, don't just claim it. Capture `EXPLAIN (ANALYZE, BUFFERS)` before and after creating `idx_waste_estab_date` on your seeded 400k-row table. A Seq Scan at ~180ms becoming an Index Scan at ~2ms is one screenshot and it wins marks in the Testing Results section (Req. 14).

5.2 Views and materialised views

```sql
-- Canonical per-establishment daily rollup, normalised by capacity
CREATE VIEW v_daily_waste AS
SELECT e.establishment_id, e.name, e.zone_id, ec.kind,
       w.occurred_on,
       SUM(w.quantity_kg)                                  AS total_waste_kg,
       SUM(w.quantity_kg) FILTER (WHERE w.was_edible)      AS edible_waste_kg,
       SUM(w.quantity_kg) FILTER (WHERE dr.is_diversion)   AS diverted_kg,
       SUM(w.cost_estimate)                                AS cost_lost,
       SUM(w.co2e_estimate)                                AS co2e_kg,
       ROUND(SUM(w.quantity_kg) / e.capacity, 4)           AS waste_kg_per_capita
FROM waste_event w
JOIN establishment e ON e.establishment_id = w.establishment_id
JOIN establishment_category ec ON ec.category_id = e.category_id
JOIN disposal_route dr ON dr.route_id = w.route_id
GROUP BY e.establishment_id, e.name, e.zone_id, ec.kind, w.occurred_on, e.capacity;

-- Over-preparation: the single most actionable metric in the system
CREATE VIEW v_preparation_efficiency AS
SELECT ss.establishment_id, ss.service_date, ss.slot,
       SUM(pb.prepared_qty_kg)                                  AS prepared_kg,
       SUM(cr.served_qty_kg)                                    AS served_kg,
       SUM(cr.leftover_qty_kg + cr.plate_waste_kg)              AS unconsumed_kg,
       ROUND(100.0 * SUM(cr.served_qty_kg)
             / NULLIF(SUM(pb.prepared_qty_kg),0), 2)            AS utilisation_pct,
       ss.expected_headcount, ss.actual_headcount,
       ROUND(100.0 * ss.actual_headcount
             / NULLIF(ss.expected_headcount,0), 2)              AS turnout_pct
FROM service_session ss
```

```sql
JOIN preparation_batch pb ON pb.session_id = ss.session_id
JOIN consumption_record cr ON cr.batch_id = pb.batch_id
WHERE ss.is_closed
GROUP BY ss.session_id, ss.establishment_id, ss.service_date, ss.slot,
         ss.expected_headcount, ss.actual_headcount;

-- Redistribution funnel
CREATE VIEW v_redistribution_performance AS
SELECT sl.establishment_id,
       COUNT(*)                                                    AS listings,
       SUM(sl.offered_qty_kg)                                      AS offered_kg,
       SUM(sl.claimed_qty_kg)                                      AS claimed_kg,
       ROUND(100.0*SUM(sl.claimed_qty_kg)/NULLIF(SUM(sl.offered_qty_kg),0),2) AS rescue_rate_pct,
       AVG(EXTRACT(EPOCH FROM (rc.collected_at - sl.created_at))/3600) AS avg_hours_to_pickup
FROM surplus_listing sl
LEFT JOIN redistribution_claim rc
       ON rc.listing_id = sl.listing_id AND rc.status = 'COLLECTED'
GROUP BY sl.establishment_id;

-- Expensive zone rollup -> materialise, refresh nightly
CREATE MATERIALIZED VIEW mv_zone_monthly AS
SELECT z.zone_id, z.zone_name, ec.kind,
       date_trunc('month', w.occurred_on)::date AS month,
       SUM(w.quantity_kg) AS waste_kg,
       SUM(w.cost_estimate) AS cost_lost,
       COUNT(DISTINCT w.establishment_id) AS reporting_estabs
FROM waste_event w
JOIN establishment e ON e.establishment_id = w.establishment_id
JOIN establishment_category ec ON ec.category_id = e.category_id
JOIN zone z ON z.zone_id = e.zone_id
GROUP BY z.zone_id, z.zone_name, ec.kind, 2;

CREATE UNIQUE INDEX ON mv_zone_monthly (zone_id, kind, month);   -- enables CONCURRENTLY
-- REFRESH MATERIALIZED VIEW CONCURRENTLY mv_zone_monthly;
```

5.3 Stored procedures and functions

```sql
-- 1) Reusable metric: waste intensity for any establishment / window
CREATE OR REPLACE FUNCTION fn_waste_intensity(p_estab INT, p_from DATE, p_to DATE)
RETURNS NUMERIC LANGUAGE plpgsql STABLE AS $$
DECLARE v_waste NUMERIC; v_served NUMERIC;
BEGIN
  SELECT COALESCE(SUM(quantity_kg),0) INTO v_waste
    FROM waste_event WHERE establishment_id = p_estab
      AND occurred_on BETWEEN p_from AND p_to;
  SELECT COALESCE(SUM(cr.served_qty_kg),0) INTO v_served
    FROM consumption_record cr
    JOIN preparation_batch pb ON pb.batch_id = cr.batch_id
    JOIN service_session ss  ON ss.session_id = pb.session_id
   WHERE ss.establishment_id = p_estab AND ss.service_date BETWEEN p_from AND p_to;
  IF v_served = 0 THEN RETURN NULL; END IF;
  RETURN ROUND(v_waste / v_served, 4);
END $$;
```

```
-- 2) The core write procedure: closing a service session.
--    Computes leftovers, deducts inventory, auto-creates waste rows,
--    and auto-lists still-edible surplus. All-or-nothing.
CREATE OR REPLACE PROCEDURE sp_close_service_session(
  p_session BIGINT, p_actual_headcount INT, p_user INT)
LANGUAGE plpgsql AS $$
DECLARE r RECORD; v_estab INT; v_reason INT; v_route INT;
BEGIN
  SELECT establishment_id INTO v_estab FROM service_session
    WHERE session_id = p_session FOR UPDATE;
  IF NOT FOUND THEN RAISE EXCEPTION 'Session % not found', p_session; END IF;

  SELECT reason_id INTO v_reason FROM waste_reason WHERE reason_code='OVER_PREPARATION';
  SELECT route_id  INTO v_route  FROM disposal_route WHERE route_name='COMPOST';

  UPDATE service_session
     SET actual_headcount = p_actual_headcount, is_closed = TRUE, closed_at = now()
   WHERE session_id = p_session;

  FOR r IN
    SELECT pb.batch_id, pb.dish_id, cr.leftover_qty_kg, cr.plate_waste_kg,
           d.dish_name, d.is_vegetarian
      FROM preparation_batch pb
      JOIN consumption_record cr ON cr.batch_id = pb.batch_id
      JOIN dish d ON d.dish_id = pb.dish_id
     WHERE pb.session_id = p_session
  LOOP
    -- plate waste is never edible
    IF r.plate_waste_kg > 0 THEN
      INSERT INTO waste_event(establishment_id, session_id, reason_id, route_id, stage,
                              quantity_kg, was_edible, occurred_on)
      VALUES (v_estab, p_session, v_reason, v_route, 'POST_CONSUMER_PLATE',
              r.plate_waste_kg, FALSE, CURRENT_DATE);
    END IF;

    -- untouched leftovers are edible -> offer for rescue instead of dumping
    IF r.leftover_qty_kg > 0 THEN
      INSERT INTO surplus_listing(establishment_id, session_id, description,
             offered_qty_kg, is_cooked, is_vegetarian, available_from, expires_at)
      VALUES (v_estab, p_session,
              format('%s kg surplus %s, freshly prepared, unserved',
                     r.leftover_qty_kg, r.dish_name),
              r.leftover_qty_kg, TRUE, r.is_vegetarian, now(), now() + interval '4 hours');
    END IF;
  END LOOP;
END $$;

-- 3) Concurrency-safe claim (called inside the transaction of 5.5)
CREATE OR REPLACE FUNCTION fn_claim_surplus(
  p_listing BIGINT, p_recipient INT, p_qty NUMERIC, p_score NUMERIC)
RETURNS BIGINT LANGUAGE plpgsql AS $$
DECLARE v_avail NUMERIC; v_claim BIGINT;
BEGIN
  SELECT offered_qty_kg - claimed_qty_kg INTO v_avail
    FROM surplus_listing WHERE listing_id = p_listing
```

```
      AND status IN ('OPEN','PARTIALLY_CLAIMED')
    FOR UPDATE;                        -- <-- serialises competing claimants

  IF NOT FOUND THEN RAISE EXCEPTION 'Listing % not claimable', p_listing; END IF;
  IF p_qty > v_avail THEN
     RAISE EXCEPTION 'Only % kg available, % kg requested', v_avail, p_qty;
  END IF;

  INSERT INTO redistribution_claim(listing_id, recipient_id, claimed_qty_kg, match_score, status)
  VALUES (p_listing, p_recipient, p_qty, p_score, 'APPROVED')
  RETURNING claim_id INTO v_claim;

  UPDATE surplus_listing
     SET claimed_qty_kg = claimed_qty_kg + p_qty,
         status = CASE WHEN claimed_qty_kg + p_qty >= offered_qty_kg
                       THEN 'FULLY_CLAIMED'::listing_status
                       ELSE 'PARTIALLY_CLAIMED'::listing_status END
   WHERE listing_id = p_listing;

  RETURN v_claim;
END $$;
```

5.4 Triggers

```
-- T1: cross-table integrity a CHECK constraint cannot express
CREATE OR REPLACE FUNCTION trg_validate_consumption() RETURNS TRIGGER AS $$
DECLARE v_prepared NUMERIC;
BEGIN
  SELECT prepared_qty_kg INTO v_prepared FROM preparation_batch WHERE batch_id = NEW.batch_id;
  IF NEW.served_qty_kg + NEW.leftover_qty_kg > v_prepared + 0.001 THEN
    RAISE EXCEPTION 'served (%) + leftover (%) exceeds prepared (%) for batch %',
      NEW.served_qty_kg, NEW.leftover_qty_kg, v_prepared, NEW.batch_id;
  END IF;
  IF NEW.plate_waste_kg > NEW.served_qty_kg THEN
    RAISE EXCEPTION 'plate waste cannot exceed served quantity';
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER tr_consumption_balance
BEFORE INSERT OR UPDATE ON consumption_record
FOR EACH ROW EXECUTE FUNCTION trg_validate_consumption();

-- T2: derive cost + carbon automatically, so the app can never write wrong economics
CREATE OR REPLACE FUNCTION trg_enrich_waste() RETURNS TRIGGER AS $$
BEGIN
  IF NEW.food_id IS NOT NULL THEN
    SELECT NEW.quantity_kg * fi.avg_cost_per_unit,
           NEW.quantity_kg * COALESCE(ef.kg_co2e_per_kg, 2.5)
      INTO NEW.cost_estimate, NEW.co2e_estimate
      FROM food_item fi
      LEFT JOIN emission_factor ef ON ef.food_category_id = fi.food_category_id
     WHERE fi.food_id = NEW.food_id;
  END IF;
```

```
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER tr_waste_enrich BEFORE INSERT ON waste_event
FOR EACH ROW EXECUTE FUNCTION trg_enrich_waste();

-- T3: inventory ledger stays consistent when a lot is consumed or wasted
CREATE OR REPLACE FUNCTION trg_deduct_lot() RETURNS TRIGGER AS $$
BEGIN
  IF NEW.lot_id IS NOT NULL THEN
    UPDATE inventory_lot
       SET remaining_qty = remaining_qty - NEW.quantity_kg
     WHERE lot_id = NEW.lot_id;
    IF (SELECT remaining_qty FROM inventory_lot WHERE lot_id = NEW.lot_id) < 0 THEN
      RAISE EXCEPTION 'Waste of % kg exceeds remaining stock in lot %', NEW.quantity_kg, NEW.lot_id;
    END IF;
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER tr_lot_deduct AFTER INSERT ON waste_event
FOR EACH ROW EXECUTE FUNCTION trg_deduct_lot();

-- T4: generic audit trail (attach to establishment, surplus_listing, redistribution_claim)
CREATE OR REPLACE FUNCTION trg_audit() RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO audit_log(table_name, record_pk, action, old_row, new_row,
                        changed_by)
  VALUES (TG_TABLE_NAME,
          COALESCE(NEW.*::text, OLD.*::text),
          LEFT(TG_OP,1),
          CASE WHEN TG_OP <> 'INSERT' THEN to_jsonb(OLD) END,
          CASE WHEN TG_OP <> 'DELETE' THEN to_jsonb(NEW) END,
          NULLIF(current_setting('app.current_user_id', true),'')::INT);
  RETURN COALESCE(NEW, OLD);
END $$ LANGUAGE plpgsql;

CREATE TRIGGER tr_audit_listing
AFTER INSERT OR UPDATE OR DELETE ON surplus_listing
FOR EACH ROW EXECUTE FUNCTION trg_audit();
```

> The `current_setting('app.current_user_id')` trick: your Node data layer runs `SET LOCAL app.current_user_id = $1` at the start of every request transaction. The audit trigger then attributes rows to real users without passing the user id into every INSERT. Small detail, very good viva answer.

5.5 Transactions

The claim flow is the showcase. In `app/api/claims/route.ts`:

```
const client = await pool.connect();
try {
  await client.query('BEGIN');
  await client.query('SET LOCAL app.current_user_id = $1', [session.userId]);
  const { rows } = await client.query(
    'SELECT fn_claim_surplus($1,$2,$3,$4) AS claim_id',
```

```
    [listingId, recipientId, qty, matchScore]
  );
  await client.query(
    `INSERT INTO waste_event(establishment_id, session_id, reason_id, route_id,
                             stage, quantity_kg, was_edible, occurred_on)
     SELECT establishment_id, session_id,
            (SELECT reason_id FROM waste_reason WHERE reason_code='REDISTRIBUTED'),
            (SELECT route_id  FROM disposal_route WHERE route_name='DONATION'),
            'PRE_CONSUMER_PREP', $2, TRUE, CURRENT_DATE
       FROM surplus_listing WHERE listing_id = $1`,
    [listingId, qty]
  );
  await client.query('COMMIT');
} catch (e) {
  await client.query('ROLLBACK');
  throw e;
} finally { client.release(); }
```

Test to demo (Req. 14, testing results): open two `psql` sessions, both `BEGIN`, both claim the same last 5 kg. Session 2 blocks on `FOR UPDATE`, then fails with *"Only 0 kg available"* after session 1 commits. Screenshot both terminals side by side. That is a concurrency proof, not a claim.

5.6 Full-text search

```
-- rank food items by relevance, with trigram fallback for typos
SELECT food_id, canonical_name,
       ts_rank(search_doc, plainto_tsquery('english', $1)) AS rank
FROM food_item
WHERE search_doc @@ plainto_tsquery('english', $1)
   OR canonical_name % $1                      -- pg_trgm similarity operator
ORDER BY rank DESC NULLS LAST, similarity(canonical_name, $1) DESC
LIMIT 10;
```

5.7 Vector similarity search

```
-- Match a surplus listing to the best recipients (cosine distance <=>)
SELECT r.recipient_id, r.org_name,
       1 - (r.need_embedding <=> sl.desc_embedding) AS similarity,
       r.daily_capacity_kg
FROM surplus_listing sl
CROSS JOIN LATERAL (
  SELECT * FROM recipient_org r2
   WHERE r2.zone_id IN (SELECT zone_id FROM zone WHERE zone_id = $2)
     AND (NOT sl.is_cooked OR r2.accepts_cooked)
     AND r2.daily_capacity_kg >= sl.offered_qty_kg * 0.5
   ORDER BY r2.need_embedding <=> sl.desc_embedding
   LIMIT 5
) r
WHERE sl.listing_id = $1;

-- Root-cause clustering: 10 most semantically similar past waste incidents
SELECT w.waste_id, w.notes, wr.reason_code,
       1 - (w.notes_embedding <=> $1::vector) AS similarity
FROM waste_event w JOIN waste_reason wr USING (reason_id)
```

```
WHERE w.notes_embedding IS NOT NULL
ORDER BY w.notes_embedding <=> $1::vector
LIMIT 10;
```

The `<=>` operator is cosine distance; `1 - distance` is similarity. **Hybrid ranking is the honest approach:** `final_score = 0.6 × cosine + 0.25 × capacity_fit + 0.15 × proximity`. Say so in the report — pure vector ranking would happily send 40 kg of chicken biryani to a vegetarian shelter with 5 kg capacity.

5.8 Window functions, CTEs, recursive CTE

```
-- Recursive: roll a metric up an arbitrary-depth zone tree
WITH RECURSIVE subtree AS (
  SELECT zone_id, parent_zone_id, zone_name, 0 AS depth
    FROM zone WHERE zone_id = $1
  UNION ALL
  SELECT z.zone_id, z.parent_zone_id, z.zone_name, s.depth + 1
    FROM zone z JOIN subtree s ON z.parent_zone_id = s.zone_id
)
SELECT s.zone_name, s.depth, COALESCE(SUM(w.quantity_kg),0) AS waste_kg
FROM subtree s
LEFT JOIN establishment e ON e.zone_id = s.zone_id
LEFT JOIN waste_event w ON w.establishment_id = e.establishment_id
   AND w.occurred_on >= CURRENT_DATE - 30
GROUP BY s.zone_name, s.depth ORDER BY s.depth, waste_kg DESC;

-- Anomaly detection entirely in SQL (no ML needed)
WITH daily AS (
  SELECT establishment_id, occurred_on, SUM(quantity_kg) AS kg
  FROM waste_event GROUP BY 1,2
), stats AS (
  SELECT *,
    AVG(kg)    OVER w AS roll_mean,
    STDDEV_SAMP(kg) OVER w AS roll_sd
  FROM daily
  WINDOW w AS (PARTITION BY establishment_id ORDER BY occurred_on
               ROWS BETWEEN 28 PRECEDING AND 1 PRECEDING)
)
SELECT establishment_id, occurred_on, kg, ROUND(roll_mean,2) AS expected,
       ROUND((kg - roll_mean)/NULLIF(roll_sd,0), 2) AS z_score
FROM stats
WHERE roll_sd > 0 AND (kg - roll_mean)/roll_sd > 2.5
ORDER BY z_score DESC;

-- Peer benchmarking: rank within your own category, not against everyone
SELECT e.name, ec.category_name,
       ROUND(SUM(w.quantity_kg)/e.capacity, 3) AS kg_per_capita,
       RANK()        OVER (PARTITION BY ec.category_id ORDER BY SUM(w.quantity_kg)/e.capacity DESC) AS worst_ran
       PERCENT_RANK() OVER (PARTITION BY ec.category_id ORDER BY SUM(w.quantity_kg)/e.capacity)     AS pct_rank
FROM waste_event w
JOIN establishment e USING (establishment_id)
JOIN establishment_category ec USING (category_id)
WHERE w.occurred_on >= CURRENT_DATE - 90
GROUP BY e.establishment_id, e.name, e.capacity, ec.category_id, ec.category_name;
```

5.9 Partitioning

`waste_event` is `PARTITION BY RANGE (occurred_on)` with monthly children. Show `EXPLAIN` on a single-month query proving **partition pruning** (only one child scanned). Add a `DO` block that pre-creates the next 12 months' partitions.

6. MongoDB design (Req. 4 — document the data model, indexes, retrieval)

6.1 Collections

`waste_submissions` — the reason Mongo exists here. Schema varies by `estab_kind`.

```
// Restaurant submission
{ _id: ObjectId("..."), establishment_id: 41, estab_kind: "COMMERCIAL",
  submitted_at: ISODate("2026-03-14T22:10:00Z"), service_date: "2026-03-14",
  submitted_by: 88, sync_status: "SYNCED", sql_waste_ids: [90211, 90212],
  payload: {
    slot: "DINNER", covers_served: 187,
    dish_waste: [
      { dish_name: "Veg Biryani", portions_prepared: 90, portions_sold: 71,
        returned_plates: 22, avg_plate_leftover_g: 85, reason_text: "party cancelled at 8pm" },
      { dish_name: "Paneer Butter Masala", portions_prepared: 40, portions_sold: 39,
        returned_plates: 3, avg_plate_leftover_g: 40 }
    ],
    buffet_counter_leftovers_kg: 6.2,
    raw_spoilage: [{ item: "tomatoes", kg: 3.1, cause: "fungal, monsoon humidity" }]
  }
}

// Household submission — completely different shape, same collection
{ _id: ObjectId("..."), establishment_id: 502, estab_kind: "RESIDENTIAL",
  submitted_at: ISODate("2026-03-14T20:02:00Z"), service_date: "2026-03-14",
  sync_status: "PENDING",
  payload: {
    bins: { vegetable_peels_kg: 0.9, cooked_leftovers_kg: 0.4, expired_packaged_kg: 0.2 },
    segregated: true, occupants_present: 4,
    photo_url: "s3://.../bin.jpg",
    free_text: "cooked extra rice for guests who didn't come"
  }
}
```

`inspection_reports` — nested checklists, variable sections, inspector free text, photos. **`bin_readings`** — time-series: `{ establishment_id, bin_id, ts, weight_kg, fill_pct }`. Use a Mongo **time-series collection** (`timeseries: {timeField:'ts', metaField:'bin_id'}`) — that alone is a legitimate "NoSQL feature" talking point. **`establishment_meta`** — operating hours per weekday, seasonal notes, tags, arbitrary attributes that differ per kind. **`activity_stream`** — append-only UI/user events, capped collection.

6.2 Indexes and validation

```
db.waste_submissions.createIndex({ establishment_id: 1, service_date: -1 });
db.waste_submissions.createIndex({ sync_status: 1, submitted_at: 1 });   // ETL queue
db.waste_submissions.createIndex({ "payload.dish_waste.dish_name": 1 }); // multikey
db.waste_submissions.createIndex({ "payload.free_text": "text" });       // text index
db.bin_readings.createIndex({ bin_id: 1, ts: -1 });
```

```
// Schema validation — proves you didn't use Mongo just to avoid discipline
db.createCollection("waste_submissions", { validator: { $jsonSchema: {
  bsonType: "object",
  required: ["establishment_id","estab_kind","service_date","payload","sync_status"],
  properties: {
    establishment_id: { bsonType: "int" },
    estab_kind: { enum: ["COMMERCIAL","EDUCATIONAL","ORGANIZATIONAL",
                         "RESIDENTIAL","RECREATIONAL","RETAIL_FOOD","HEALTHCARE"] },
    sync_status: { enum: ["PENDING","SYNCED","FAILED"] },
    payload: { bsonType: "object" }
  }
}}});
```

6.3 Aggregation pipelines (Req. 7: “NoSQL aggregation”)

```
// Which dishes are wasted most across all restaurants, last 30 days
db.waste_submissions.aggregate([
  { $match: { estab_kind: "COMMERCIAL",
              submitted_at: { $gte: new Date(Date.now() - 30*864e5) } } },
  { $unwind: "$payload.dish_waste" },
  { $group: {
      _id: { $toLower: "$payload.dish_waste.dish_name" },
      total_returned: { $sum: "$payload.dish_waste.returned_plates" },
      est_waste_kg: { $sum: { $divide: [
        { $multiply: ["$payload.dish_waste.returned_plates",
                      "$payload.dish_waste.avg_plate_leftover_g"] }, 1000 ] } },
      restaurants: { $addToSet: "$establishment_id" },
      avg_sell_through: { $avg: { $divide: [
        "$payload.dish_waste.portions_sold", "$payload.dish_waste.portions_prepared"] } }
  }},
  { $addFields: { restaurant_count: { $size: "$restaurants" } } },
  { $match: { restaurant_count: { $gte: 3 } } },   // systemic, not one bad kitchen
  { $sort: { est_waste_kg: -1 } }, { $limit: 15 },
  { $project: { restaurants: 0 } }
]);

// $facet: one round trip powers an entire dashboard
db.waste_submissions.aggregate([
  { $match: { service_date: { $gte: "2026-03-01" } } },
  { $facet: {
      by_kind:   [{ $group: { _id: "$estab_kind", submissions: { $sum: 1 } } }],
      compliance:[{ $group: { _id: "$sync_status", n: { $sum: 1 } } }],
      household_bins: [
        { $match: { estab_kind: "RESIDENTIAL" } },
        { $group: { _id: null,
            peels: { $sum: "$payload.bins.vegetable_peels_kg" },
            cooked:{ $sum: "$payload.bins.cooked_leftovers_kg" },
            packaged:{ $sum: "$payload.bins.expired_packaged_kg" } } }
      ]
  }}
]);
```

6.4 The ETL bridge (Mongo → Postgres)

A Next.js route handler (or cron) that runs every 10 minutes:

1. `find({sync_status:'PENDING'}).limit(200)`
2. For each doc, normalise by `estab_kind` into `waste_event` rows: resolve `dish_name` → `food_id` via **vector nearest-neighbour** on `food_item.name_embedding` (fall back to trigram, else `NULL` + flag for review).
3. Insert into Postgres inside one transaction; the enrich trigger fills cost and CO2e.
4. `updateOne({_id}, {$set: {sync_status:'SYNCED', sql_waste_ids: [...]}})`.
5. On failure: `sync_status:'FAILED'` with the error message; a small admin page lists failures.

This ETL is the single best thing to demo. It is where all three paradigms touch: document → vector lookup → relational write → trigger-derived economics. If you only get to show one thing to the examiner, show this.

7. Query catalogue (Req. 6 — JOIN, GROUP BY, HAVING, ORDER BY, aggregates, subqueries)

Build these 22 as named, tested queries in `db/queries/`. Each maps to a UI element, so none of them is decoration. The right-hand column is what you tick off in the report.

#	Query	SQL features shown
1	Zone waste summary for a date range, by establishment kind	JOIN ×4, GROUP BY, SUM, date filter
2	Top 10 wasted food items in a zone, with % of zone total	window `SUM() OVER ()`, subquery, ORDER BY, LIMIT
3	Establishments whose 90-day per-capita waste exceeds their category average	**correlated subquery** + HAVING
4	Waste split by stage (pre vs post consumer) per kind	`FILTER (WHERE ...)`, GROUP BY ROLLUP
5	Chronic over-preparers: utilisation < 75% on ≥ 10 sessions	view + HAVING COUNT(*) >= 10
6	Turnout vs preparation correlation per establishment	`CORR()` aggregate
7	Day-of-week waste profile	`EXTRACT(DOW ...)`, GROUP BY, ORDER BY
8	Waste on event days vs non-event days	M:N join through `zone_event`, CASE aggregation
9	Lots expiring within 24h with remaining stock, ranked by value at risk	partial index, ORDER BY computed expr
10	Rescue rate funnel: offered → claimed → collected	LEFT JOIN chain, conditional aggregates

#	Query	SQL features shown
11	Recipients with unmet capacity while surplus expired unclaimed in same zone	`EXCEPT` / anti-join `NOT EXISTS`
12	Median hours from listing to pickup, per establishment kind	`PERCENTILE_CONT` ordered-set aggregate
13	Diversion rate (non-landfill %) trend by month	materialised view + window
14	Monetary loss and CO2e per establishment, last quarter	SUM on derived columns
15	Anomalous waste days (z-score > 2.5)	window frame `28 PRECEDING`
16	Rolling 7-day average waste per establishment	`AVG() OVER (ORDER BY ... ROWS 6 PRECEDING)`
17	Peer rank & percentile within category	`RANK()`, `PERCENT_RANK()` PARTITION BY
18	Recursive zone rollup to any depth	`WITH RECURSIVE`
19	Ingredient-level exposure: which raw items are implicated in most wasted dishes	M:N via `dish_ingredient`, GROUP BY, HAVING
20	Suppliers whose deliveries most often end in spoilage waste	multi-hop JOIN procurement→lot→waste, HAVING
21	Forecast accuracy: MAE/ MAPE per model version and slot	JOIN forecast↔actual, ABS, AVG
22	Semantic top-5 recipient match for a listing	pgvector `<=>` + LATERAL

Query 3 written out, since correlated subqueries are the one examiners always ask about:

```
SELECT e.name, ec.category_name,
       ROUND(SUM(w.quantity_kg)/e.capacity, 3) AS kg_per_capita
FROM waste_event w
JOIN establishment e USING (establishment_id)
JOIN establishment_category ec USING (category_id)
WHERE w.occurred_on >= CURRENT_DATE - 90
GROUP BY e.establishment_id, e.name, e.capacity, ec.category_name, ec.category_id
HAVING SUM(w.quantity_kg)/e.capacity > (
  SELECT AVG(sub.per_cap) FROM (
    SELECT SUM(w2.quantity_kg)/e2.capacity AS per_cap
    FROM waste_event w2 JOIN establishment e2 USING (establishment_id)
    WHERE e2.category_id = ec.category_id            -- correlation
      AND w2.occurred_on >= CURRENT_DATE - 90
    GROUP BY e2.establishment_id, e2.capacity
  ) sub)
```

```
ORDER BY kg_per_capita DESC;
```

8. Application design (Next.js — Req. 9, 10)

8.1 Route map

```
app/
├── (auth)/login/page.tsx
├── (estab)/                          role: ESTAB_MANAGER, ESTAB_STAFF
│   ├── dashboard/page.tsx            today's KPIs, expiry alerts, anomaly banner
│   ├── inflow/page.tsx               procurement + lines form (multi-row txn)
│   ├── service/page.tsx          open session → batches → close session (calls sp_close_service_session)
│   ├── waste/page.tsx                log waste; free-text notes get embedded
│   ├── inventory/page.tsx            lots, remaining, expiry countdown
│   └── surplus/page.tsx              list surplus, see suggested recipients w/ match scores
├── (recipient)/                      role: RECIPIENT_AGENT
│   ├── feed/page.tsx                 semantic-ranked available surplus
│   └── claims/page.tsx               claim, mark collected
├── (authority)/                      role: ZONE_ADMIN, AUDITOR
│   ├── zone/page.tsx                 map + recursive zone rollup + kind breakdown
│   ├── benchmarking/page.tsx         peer ranks, chronic over-preparers
│   ├── analytics/page.tsx            trends, day-of-week, event impact, diversion rate
│   ├── insights/page.tsx             similar-incident clustering (vector), FTS search
│   ├── forecasts/page.tsx            predictions + accuracy table
│   └── etl/page.tsx                  Mongo sync queue, failures, manual re-run
└── api/
    ├── auth/[login|logout|me]/route.ts
    ├── establishments/route.ts              GET, POST
    ├── establishments/[id]/route.ts         GET, PATCH, DELETE  (soft delete)
    ├── procurement/route.ts                 POST (transaction: header + N lines + N lots)
    ├── sessions/route.ts                    POST
    ├── sessions/[id]/close/route.ts         POST → CALL sp_close_service_session
    ├── waste/route.ts                       POST, GET (filters)
    ├── submissions/route.ts                 POST → MongoDB
    ├── etl/sync/route.ts                    POST → Mongo→PG bridge
    ├── surplus/route.ts                     GET (vector-ranked), POST
    ├── claims/route.ts                      POST (the transaction demo)
    ├── analytics/[metric]/route.ts          the 22 queries, parameterised
    ├── search/route.ts                      FTS + trigram
    ├── similar/route.ts                     vector KNN over waste notes
    └── forecast/route.ts                    proxies FastAPI, persists to demand_forecast
```

8.2 Data-access layer (one file, everything goes through it)

lib/db.ts

```
import { Pool } from 'pg';
export const pool = new Pool({ connectionString: process.env.DATABASE_URL, max: 10 });

export async function withTx<T>(userId: number | null, fn: (c) => Promise<T>) {
  const c = await pool.connect();
  try {
    await c.query('BEGIN');
    if (userId) await c.query('SET LOCAL app.current_user_id = $1', [String(userId)]);
    const out = await fn(c);
```

```
    await c.query('COMMIT');
    return out;
  } catch (e) { await c.query('ROLLBACK'); throw e; }
  finally { c.release(); }
}
```

Every query uses `$1`, `$2` placeholders. **Never** template-literal user input into SQL — that's the injection point examiners probe (Req. 13).

8.3 UI components worth building (Req. 9 asks for forms, tables, search, filtering, dashboards)

- KPI cards: waste kg, ₹ lost, kg CO2e, diversion %, rescue rate
- Recharts: stacked area (waste by stage over time), bar (top foods), grouped bar (kind comparison), line (forecast vs actual)
- Filterable/sortable/paginated waste table with server-side filtering (date, kind, zone, stage, reason)
- Search box hitting `/api/search` (FTS) with typo tolerance
- The surplus match panel: listing on the left, ranked recipients on the right with the similarity score *shown as a number* — visible proof the vector layer does something
- Expiry countdown list with colour bands

9. Auth, roles, security (Req. 11, 12, 13)

- **Hashing:** `bcryptjs`, cost 12. Never store, log, or return `password_hash`.
- **Session:** JWT signed with `jose`, stored in an `httpOnly`, `sameSite=lax`, `secure` cookie. Payload: `{userId, role, establishmentId, recipientId}`.
- **Middleware:** `middleware.ts` gates route groups by role. Every API handler *re-checks* — middleware alone is not authorisation.
- **Tenant scoping:** an `ESTAB_MANAGER` can only read/write rows for their own `establishment_id`. Enforce in the WHERE clause of every query, not just in the UI. Optional stretch: Postgres **Row-Level Security** policies keyed on `app.current_user_id` — a genuinely impressive addition if you have time.
- **Validation in three layers** (Req. 12): `zod` at the API boundary → CHECK constraints in DDL → triggers for cross-table rules. Show all three failing on the same bad input in your test evidence.
- **Roles:** 5 defined (requirement is minimum 2). `AUDITOR` is read-only — a nice way to show a role that has *no* write path.
- `.env.example` committed, `.env*` git-ignored, no credential anywhere in source.

10. Seed data (do this in Week 2 — nothing works without it)

You cannot get real data for 500 households. Generate it, and be transparent in the report that it is synthetic with documented distributions. Target volumes:

Table	Rows
zone	1 sector → 6 blocks → 24 streets
establishment	~60 non-residential + 480 residential units
food_item	150
dish	~400

Table	Rows
procurement / procurement_line	25k / 90k
inventory_lot	90k
service_session	~40k (14 months)
preparation_batch / consumption_record	160k / 160k
waste_event	**~450k**
surplus_listing / redistribution_claim	12k / 7k
Mongo `waste_submissions`	~60k

Generator (`scripts/seed.py`) rules that make the data *analysable* rather than random noise — this matters, because flat random data produces dashboards with nothing in them:

1. Base demand per establishment ∝ capacity, with kind-specific multipliers.
2. **Weekly seasonality:** colleges drop 60% on Sunday; restaurants peak Fri–Sat; offices near-zero on weekends.
3. **Event effects:** exam periods → mess attendance up, plate waste up; festivals → restaurant volume +40%; vacations → institutional demand collapses.
4. **Planted signals you will later "discover":** make establishment #17 (a school) chronically over-prepare by 25%; make one supplier's produce spoil 3× more often; make monsoon months (Jun–Sep) increase `PRE_CONSUMER_SPOILAGE`. Your analytics pages then find real patterns, and your demo has a story.
5. **Noise:** Gaussian ~15% CV, plus 2% of days with genuine anomalies so query 15 returns something.
6. **Messy food names** in Mongo submissions on purpose (`Veg Biriyani`, `veg biryani`, `VEG. BIRYANI`) so the vector normalisation step visibly earns its place.
7. Free-text `notes` drawn from ~40 templates across 8 latent causes, so similarity clustering has real structure to find.

Run the embedding backfill once after seeding: `scripts/backfill_embeddings.py` batches `food_item`, `recipient_org`, `surplus_listing`, and a sample of `waste_event.notes` through MiniLM and `UPDATE`s the vector columns.

11. Forecasting layer (keep it small — it is not what you're graded on)

Scope: predict `prepared_qty_kg` needed per (establishment, dish, date, slot).

- **Features:** lag-1/7/14 demand, rolling 7/28-day mean, day-of-week, slot, is_event, event impact, expected_headcount, month, establishment kind, capacity, dish popularity share.
- **Split:** time-based (train ≤ month 11, validate month 12, test months 13–14). Never random-shuffle a time series.
- **Models:** three, not five — seasonal naive baseline, ridge regression, LightGBM. Report MAE / RMSE / MAPE. Three is enough to justify a choice; five is padding.
- **Recommendation:** `recommended = predicted × (1 + buffer)` where buffer comes from the **quantile loss** model at q=0.85 rather than a flat percentage. Explain in the report that a fixed buffer ignores variance, and quantile regression prices the asymmetry (under-preparing hurts more than over-preparing, up to a point).
- **Explainability:** store the top drivers in `demand_forecast.explanation` JSONB and render them: *"Predicted 62 kg — 8% below your 4-week Tuesday average, because attendance is forecast down 12% during exam break."*
- **The DB part that actually earns marks:** `demand_forecast` joined against real outcomes to compute accuracy in SQL (query 21), and a `forecast_model` table where exactly one version

can be active (enforced by that partial unique index in 4.3).

12. Timeline and team split

Two members, both touching database *and* application (Req. 2). Split **by vertical slice**, not by layer.

Member A — "Lifecycle & Integrity" Postgres core schema (migrations 001–005), procurement/service/waste write paths, transactions, triggers, stored procedures, partitioning, indexing + EXPLAIN evidence, establishment & service UI, queries 1–11.

Member B — "Intelligence & Distribution" MongoDB modelling + aggregations + ETL bridge, pgvector layer + embedding service, redistribution marketplace & claim transaction, forecasting service, authority dashboards, queries 12–22.

Both write SQL, both write Next.js. Commit under your own accounts so `git log` proves it (Req. 15).

10-week plan

Week	Both	Member A	Member B
1	Docker compose up, repo skeleton, API contract agreed, ER diagram drafted	migrations 001–003	Mongo collections + validators
2	Seed generator v1 running	migrations 004–006, constraints	embedding service + backfill
3	Auth + roles + middleware working end to end	procurement & inventory API + UI	submissions API → Mongo
4	—	service session flow + `sp_close_service_session`	ETL bridge Mongo→PG with vector name resolution
5	Mid-point demo to each other; fix schema pain now, not later	triggers T1–T4, transaction tests	surplus listing + vector matching
6	—	queries 1–11 + establishment dashboards	claim transaction + recipient UI + concurrency test
7	—	indexes + partitioning + EXPLAIN evidence	Mongo aggregations + authority analytics pages
8	—	views + materialised views + FTS	forecasting service + `demand_forecast` + accuracy page
9	Freeze features. Test matrix, negative tests, screenshots		
10	Report, README, ER diagram final, demo rehearsal ×3		

If you have only 5 weeks: Weeks 1–2 as above (schema + seed + auth), Week 3 = service/waste flow + triggers + transactions, Week 4 = Mongo + ETL + 12 queries + dashboards, Week 5 = pgvec-

tor matching + report. Drop forecasting entirely; you still satisfy every numbered requirement. Cut ML before you cut database features — the rubric does not mention ML once.

13. Testing (Req. 14, 16 — “test cases”, “testing results”)

Build a table with these rows and paste actual outputs:

Negative tests — prove the database refuses bad data 1. `INSERT consumption_record` with served + leftover > prepared → trigger raises. 2. `INSERT waste_event` with `quantity_kg = -5` → CHECK violation. 3. `UPDATE surplus_listing SET claimed_qty_kg = 999` → `ck_listing_balance` violation. 4. Create `ESTAB_MANAGER` with `establishment_id = NULL` → `ck_user_scope` violation. 5. `INSERT residential_profile` with occupants < households → CHECK violation. 6. Delete a `zone` that has establishments → `ON DELETE RESTRICT`. 7. Duplicate `(establishment_id, service_date, slot)` → unique violation. 8. API call with `' OR 1=1 --` in a filter → parameterised query treats it as a literal, returns 0 rows. 9. `ESTAB_STAFF` token calling an authority endpoint → 403. 10. `AUDITOR` attempting any POST → 403.

Concurrency test: the two-terminal claim race from 5.5.

Performance test: `EXPLAIN (ANALYZE, BUFFERS)` for query 1 before/after `idx_waste_estab_date`; partition pruning proof; `SET enable_indexscan = off` comparison.

Integration test: submit a Mongo document → run ETL → assert the `waste_event` rows exist with correct trigger-derived `cost_estimate` and the doc flips to `SYNCED`.

14. Deliverables checklist (mapped to the requirements PDF)

Req.	Deliverable	Where it lives
1	Unique title, non-CRUD problem	§1
2	2 members, defined responsibilities	§12 + README
3	≥2 paradigms	Postgres + MongoDB + pgvector, §2
4	ER diagram → relational schema; Mongo data model documented	§4.1, §4.3, §6
5	DDL: CREATE/ALTER/ DROP, keys, constraints, indexes, views	`db/migrations/`, §4.3, §5.1–5.2
6	DML with JOIN/GROUP BY/HAVING/subqueries	§7
7	≥4 advanced concepts	9 delivered, §5
8	Vector/semantic search explained	§2.3, §5.7
9	Next.js interface	§8
10	API layer, env vars, no hard-coded creds	§8.2, §9
11	Auth + ≥2 roles	5 roles, §9
12	Validation at DB level, integrity	§5.4, §13
13	Security practices	§9
14	Full documentation set	write from this doc

Req.	Deliverable	Where it lives
15	Git/GitHub, individual commits, README	§12
16	Demo covering all listed items	§15
17	Originality	locality-scale framing is not a stock tutorial project
18	Beyond CRUD, justified DB choice	§2 is the answer to this

Include an **ALTER/DROP migration** on purpose (e.g. `019_add_segregation_flag.sql` adding `residential_profile.segregation_compliant`, plus dropping a genuinely unused column). Req. 5 lists ALTER and DROP explicitly and most teams forget to demonstrate them.

15. Demo script (12 minutes)

1. **Problem** (45s) — food waste as a zone-level system problem, not a kitchen problem.
2. **ER diagram + paradigm justification** (2 min) — the §2 argument. This is the highest-value part of the whole demo.
3. **Live: establishment flow** (2 min) — log procurement → open session → close session → watch `sp_close_service_session` auto-create waste rows *and* a surplus listing.
4. **Live: constraint refusal** (1 min) — try to record consumed > prepared, show the trigger error surfacing in the UI.
5. **Live: Mongo → ETL → Postgres** (2 min) — submit a household doc and a restaurant doc with different shapes, run the sync, show messy names resolved by vector lookup, show cost/CO2e appear from the trigger.
6. **Live: semantic matching + claim** (2 min) — listing, ranked recipients with scores, claim, then the two-terminal race showing the rollback.
7. **Analytics** (1.5 min) — zone rollup (recursive CTE), peer benchmarking, anomaly detection, diversion rate. Name the SQL feature behind each chart as you show it.
8. **Performance** (45s) — EXPLAIN before/after, partition pruning.
9. **Contributions + repo** (30s) — `git log --author`, README.

16. Pitfalls that sink projects like this

- **Building the UI before the seed data.** Empty dashboards feel broken and you'll redesign twice. Seed in Week 2.
- **Using an ORM.** It will hide your DDL, fight your triggers, and refuse your vector columns. Raw parameterised SQL.
- **Letting scope crawl toward maps, IoT, mobile, chatbots.** Everything in §16 of your old draft's "avoid" list still applies. One zone, one clean loop: *Record → Normalise → Analyse → Match → Redistribute → Evaluate.*
- **Treating Mongo as a second SQL database.** If your Mongo documents all have identical fields, your justification collapses and an examiner will say so. Heterogeneity by establishment type is the entire argument — keep it real.
- **Vector search as decoration.** Show the similarity numbers in the UI and show one case where keyword search fails and semantic search succeeds. Otherwise it reads as a checkbox.
- **Forgetting `ALTER/DROP`.** Explicitly listed in Req. 5.
- **Both members committing from one machine.** Req. 15 wants individual contribution evidence in the git history.
- **Over-preparing on ML.** Three models, one table, done. Every hour you spend on hyperparameter tuning is an hour not spent on the thing being graded.

17. Immediate next steps

1. Confirm the title and lock the zone scope (1 sector, ~540 establishments including residential units).
2. `docker compose up` with `pgvector/pgvector:pg16` and `mongo:7`.
3. Type migrations 001–003 and run them. Draw the ER diagram from the finished DDL, not before — the DDL is the source of truth.
4. Write `scripts/seed.py` for zones, categories, establishments, food items. Get 60 establishments in the database on day 3.
5. Then, and only then, start the Next.js app.

The one design decision to keep returning to: **every feature must be answerable as "which database concept does this demonstrate, and why is that concept the right tool here?"** If a feature has no good answer to that, cut it.

PART B — TABLE CATALOGUE, PROCESSING PIPELINES & FRONTEND

Part A gave you the schema, the DDL and the build plan. Part B is the reference layer underneath it: **why each table exists**, **how data actually turns into a prediction or an alert**, and **what each of the five people using the system sees and does**. Use Part A to build; use Part B to defend design decisions in the report and the viva.

B0. Table catalogue at a glance

39 storage objects in Parts A/B: **31 PostgreSQL tables, 5 MongoDB collections, 3 vector-bearing columns + 1 embedding-model registry.** Tiered so the system is demoable early:

Tier	Count	Meaning	Build in
T1 — Core	16	Without these there is no product	Weeks 1–4
T2 — Extended	13	What makes it “beyond CRUD”	Weeks 4–7
T3 — Intelligence	9–10	Forecasting, alerts, ops — cuttable	Weeks 7–9

(Part C adds a fourth tier, **T4 — Research Extension**, of 10 further objects — see Part D for the combined count across all four tiers.) ## B1.0 How to read this

39 storage objects total: **31 PostgreSQL tables**, **5 MongoDB collections**, **3 vector-bearing columns + 1 registry**. That sounds like a lot until you notice 9 of them are small lookup tables you seed once and never touch again.

They are tiered. Build Tier 1 first and you have a demoable system by Week 4. Tier 3 exists only if the timeline holds.

Tier	Count	Meaning
T1 — Core	16	Without these there is no product. Build weeks 1–4.
T2 — Extended	13	These are what make it “beyond CRUD”. Build weeks 4–7.
T3 — Intelligence	10	Forecasting, alerts, ops. Build weeks 7–9. Cuttable.

B1.1 GROUP A — Geography & Entities (6 tables)

#	Table	Tier	Purpose	Key columns	Why it exists as its own table
A1	`zone`	T1	The locality hierarchy: Sector → Block → Street	`zone_id` PK, `parent_zone_id` FK→self, `zone_level`, `pincode`, `centroid_lat/lng`	Self-referencing so you can aggregate at any depth with **one recursive CTE** instead of three hardcoded columns (`sector`, `block`, `street`) that would break the moment a zone gets subdivided
A2	`establishment_category`	T1	Lookup: 'Fine Dining', 'College Mess', 'Apartment Block', 'Corporate Cafeteria'…	`category_id` PK, `kind` ENUM, `category_name`, `capacity_metric`	`capacity_metric` ('seats' / 'students' / 'residents') is what makes cross-type comparison legal. Without it you'd be comparing 200 seats against 200 residents as if they were the same number
A3	`establishment`	T1	**The hub entity.** Every restaurant, college, office, apartment block, club	`establishment_id` PK, `zone_id` FK, `category_id` FK, `name`, `capacity`, `lat/lng`, `is_active`	Supertype. Every waste event, procurement, session and listing points here. One FK target, not six
A4	`commercial_profile`	T2	Subtype: restaurants, cafés, bakeries, hotels	PK=FK `establishment_id`, `cuisine_type`, `seating_capacity`, `avg_daily_covers`, `has_buffet`, `fssai_license`	`has_buffet` is a real predictor of waste. Putting it on `establishment` would leave it NULL for 500 households
A5	`institutional_profile`	T2	Subtype: schools, colleges, hostels, offices, hospitals	PK=FK, `enrolled_strength`, `mess_type`, `serves_meals_per_day`, `academic_calendar_ref`	Institutions have *enrolled* vs *present* strength — a distinction commercial venues don't have and households don't have
A6	`residential_profile`	T2	Subtype: individual houses, apartment blocks	PK=FK, `household_count`, `total_occupants`, `has_common_kitchen`, `segregation_compliant`	Lets you model 480 residential units without inventing fake "seating capacity" for each

Design note for the viva: this is *disjoint, total specialisation* — every establishment has exactly one profile row, enforced by the app + a check query. An examiner asking "why not one wide table?" gets the answer: 14 columns would be NULL for ~89% of rows (the residential units), which violates 3NF's spirit and makes every query filter on kind before it can trust a column.

B1.2 GROUP B — Identity, Access & Audit (4 tables)

#	Table	Tier	Purpose	Key columns	Notes
B1	app_user	T1	All humans in the system	user_id PK, email UQ, password_hash, role ENUM, establishment_id FK (nullable), recipient_id FK (nullable), is_active, last_login_at	The ck_user_scope CHECK enforces that a staff user *must* have an establishment and *must not* have a recipient org. **Role/tenant coherence enforced in the database, not just in middleware** — this is a strong Req. 11/12 answer
B2	user_session	T3	Refresh-token store, enables revocation	session_id PK, user_id FK, token_hash, issued_at, expires_at, revoked_at, ip, user_agent	Optional. Stateless JWT alone is fine for a course project; add this only if you want to demo "admin force-logs-out a user"
B3	audit_log	T2	Trigger-written change history	audit_id PK, table_name, record_pk, action I/U/D, old_row JSONB, new_row JSONB, changed_by FK, changed_at	Written entirely by triggers. Nothing in application code inserts here — that's the point
B4	etl_run_log	T3	One row per ETL execution	run_id PK, started_at, finished_at, docs_read, rows_written, docs_failed, unresolved_names, status, error_detail	Makes the Mongo→PG bridge observable. Powers the admin ETL page. Without it, sync failures are invisible

B1.3 GROUP C — Food Reference / Master Data (6 tables)

#	Table	Tier	Purpose	Key columns	Notes
C1	food_category	T1	Cereals, Cooked Rice Dishes, Dairy, Produce, Meat, Packaged, Beverages	food_category_id PK, category_name UQ, is_perishable, default_shelf_life_hours	default_shelf_life_hours is what lets you auto-compute inventory_lot.expires_at when a supplier gives no expiry date

#	Table	Tier	Purpose	Key columns	Notes
C2	food_item	T1	Canonical food catalogue (~150 rows)	food_id PK, food_category_id FK, canonical_name UQ, base_unit, avg_cost_per_unit, is_edible_as_is, **name_embedding vector(384)**, **search_doc tsvector**	Carries *both* the vector column (semantic name resolution) and the FTS column (keyword search). One table, two paradigms visible side by side
C3	food_alias	T2	Messy name → canonical food_id, learned by the ETL	alias_id PK, raw_name, food_id FK, resolution_method ('EXACT' / 'TRIGRAM' / 'VECTOR' / 'MANUAL'), confidence, times_seen, confirmed_by FK	**Undervalued table.** It turns your vector lookup into a cache — after the first resolution of "Veg Biriyani (Spl)" you never pay the embedding cost again. Also gives an admin screen where a human confirms low-confidence matches
C4	emission_factor	T2	kg CO2e and litres water per kg, by food category	PK=FK food_category_id, kg_co2e_per_kg, litres_water_per_kg, source_citation	Cite a real source (FAO / Poore & Nemecek 2018). Turns "18 kg wasted" into "18 kg = 47 kg CO2e", which is what makes the authority dashboard persuasive
C5	supplier	T2	Who food came from	supplier_id PK, name, zone_id FK, supplier_type, contact_phone	Enables query 20: *which suppliers' deliveries most often end in spoilage waste* — a genuinely non-obvious insight only a database can produce
C6	embedding_model	T3	Registry of which model produced which vectors	model_key PK ('minilm-l6-v2'), dimensions, similarity_metric, created_at, is_active	If you ever re-embed with a different model, every existing vector becomes meaningless. This table + a model_key column on embedded rows prevents silently mixing vector spaces

B1.4 GROUP D — Food Inflow (3 tables)

#	Table	Tier	Purpose	Key columns	Notes
D1	procurement	T2	Header: one delivery event	procurement_id PK, establishment_id FK, supplier_id FK, invoice_no, received_at, total_cost, recorded_by FK	Header/line split is the classic 3NF demonstration: supplier and date belong to the delivery, not to each item in it
D2	procurement_line	T2	One food item within a delivery	line_id PK, procurement_id FK, food_id FK, quantity, unit_cost, expiry_date	Multi-row insert inside a transaction — your simplest transaction demo
D3	inventory_lot	T2	**The stock ledger.** Trackable batch with an expiry clock	lot_id PK, establishment_id FK, food_id FK, source_line_id FK, received_qty, remaining_qty, unit_cost, received_at, expires_at	This is the table that answers *"what food is still edible but about to be lost?"* — the highest-value question in the whole system. remaining_qty is maintained by trigger, never by the app

B1.5 GROUP E — Preparation & Service (5 tables)

#	Table	Tier	Purpose	Key columns	Notes
E1	dish	T1	What an establishment actually cooks	dish_id PK, establishment_id FK, dish_name, standard_portion_g, is_vegetarian	standard_portion_g is the converter between "portions" (how kitchens think) and "kg" (how the database compares across establishment types)
E2	dish_ingredient	T2	**M:N bridge** dish ↔ food_item	composite PK (dish_id, food_id), qty_per_portion	Your required many-to-many. Also enables ingredient-level analysis: "onions appear in 60% of wasted dishes"
E3	service_session	T1	One meal service occasion	session_id PK, establishment_id FK, service_date, slot ENUM, expected_headcount, actual_headcount, is_closed, closed_at	UQ on (establishment_id, service_date, slot) — you cannot serve lunch twice on the same day. expected vs actual is the single strongest forecasting feature you have

#	Table	Tier	Purpose	Key columns	Notes
E4	preparation_batch	T1	How much of one dish was cooked for one session	batch_id PK, session_id FK, dish_id FK, prepared_qty_kg, prepared_portions, prep_cost, prepared_at	The “supply” side of every over-preparation calculation
E5	consumption_record	T1	What happened to that batch	PK=FK batch_id, served_qty_kg, leftover_qty_kg, plate_waste_kg, recorded_by FK	1:1 with batch. Separated from preparation_batch because prepared quantity is known *before* service and consumption *after* — two different write moments, two different users, two different times

The critical invariant: `served + leftover ≤ prepared`, and `plate_waste ≤ served`. This spans two tables so a CHECK constraint can’t express it — hence trigger `tr_consumption_balance`. Say exactly this when asked why you used a trigger.

B1.6 GROUP F — Waste (3 tables)

#	Table	Tier	Purpose	Key columns	Notes
F1	waste_reason	T1	Lookup: OVER_PREPARATION, LOW_TURNOUT, SPOILAGE, EXPIRY, PREP_TRIM, CUSTOMER_LEFTOVER, STORAGE_FAILURE, REDISTRIBUTED	reason_id PK, reason_code UQ, description, is_preventable	is_preventable splits “waste you can fix” (over-prep) from “waste you can’t” (vegetable peels). Every report should separate these — otherwise you’re blaming kitchens for banana skins
F2	disposal_route	T1	LANDFILL, COMPOST, BIOGAS, ANIMAL_FEED, DONATION	route_id PK, route_name UQ, is_diversion, cost_per_kg	is_diversion gives you the **diversion rate** — the metric municipalities actually report on

#	Table	Tier	Purpose	Key columns	Notes
F3	waste_event	T1	**The fact table.** ~450k rows	(waste_id, occurred_on) composite PK, establishment_id, food_id, session_id, lot_id, reason_id, route_id, stage ENUM, quantity_kg, was_edible, cost_estimate, co2e_estimate, notes, **notes_embedding vector(384)**, source_doc_id	PARTITION BY RANGE (occurred_on), monthly children. Narrow, append-mostly, heavily indexed. stage (pre-consumer prep / spoilage / expiry vs post-consumer plate / service) is the dimension that makes root-cause analysis possible at all

Why waste_event has both session_id and lot_id (both nullable): waste traces back to *either* a service occasion (leftovers, plate waste) *or* a stored batch (spoilage, expiry) — rarely both, sometimes neither (household bin logs). Nullable FKs with a documented rule beat forcing everything through one path.

B1.7 GROUP G — Redistribution (3 tables)

#	Table	Tier	Purpose	Key columns	Notes
G1	recipient_org	T1	NGOs, shelters, community fridges, animal shelters, compost units	recipient_id PK, zone_id FK, org_name UQ, org_type, need_profile TEXT, **need_embedding vector(384)**, daily_capacity_kg, accepts_cooked, pickup_window_start/end	need_profile is deliberately free text — real NGO needs don't fit dropdowns. That's exactly why the vector column is justified rather than decorative
G2	surplus_listing	T1	Edible surplus offered for rescue	listing_id PK, establishment_id FK, food_id FK, session_id FK, description TEXT, **desc_embedding vector(384)**, offered_qty_kg, claimed_qty_kg, is_cooked, is_vegetarian, available_from, expires_at, status ENUM	claimed_qty_kg ≤ offered_qty_kg CHECK + FOR UPDATE locking is your concurrency showcase. The **partial index** on OPEN listings is your index-strategy showcase

#	Table	Tier	Purpose	Key columns	Notes
G3	`redistribution_claim`	T1	A recipient claiming part of a listing	`claim_id` PK, `listing_id` FK, `recipient_id` FK, `claimed_qty_kg`, `match_score`, `status` ENUM, `requested_at`, `collected_at`	Storing `match_score` (the cosine similarity at claim time) lets you later ask *"do high-similarity matches actually get collected more often?"* — a question that validates the vector layer with real evidence

B1.8 GROUP H — Context (2 tables)

#	Table	Tier	Purpose	Key columns	Notes
H1	`locality_event`	T2	Festivals, holidays, exam periods, vacations, local events, weather episodes	`event_id` PK, `event_name`, `event_type`, `start_date`, `end_date`, `expected_impact` (-2..+2)	Without this, every forecast model will be baffled by Diwali and every anomaly detector will fire on it
H2	`zone_event`	T2	**M:N bridge** zone ↔ event	composite PK `(zone_id, event_id)`	A college exam period affects one block; a public holiday affects all. Not every event is zone-wide — hence M:N, not a column

B1.9 GROUP I — Forecasting & Evaluation (4 tables)

#	Table	Tier	Purpose	Key columns	Notes
I1	`forecast_model`	T3	Model registry	`model_version` PK ('lgbm_v3'), `algorithm`, `trained_at`, `feature_list`, `train_rows`, `val_mae`, `is_active`	`CREATE UNIQUE INDEX ... WHERE is_active` = **exactly one active model, enforced by the database.** A partial unique index used as a business rule is a genuinely elegant thing to show

#	Table	Tier	Purpose	Key columns	Notes
I2	`feature_snapshot`	T3	Materialised training/inference features	`snapshot_id` PK, `establishment_id`, `dish_id`, `target_date`, `slot`, ~18 feature columns, `actual_qty_kg` (label, nullable for future dates), `built_at`	This is your **feature store**. Building it as a table rather than recomputing features in Python guarantees training and inference see identical logic — the #1 cause of models that work in the notebook and fail in production
I3	`demand_forecast`	T3	Predictions written back to SQL	`forecast_id` PK, `establishment_id`, `dish_id`, `target_date`, `slot`, `model_version` FK, `predicted_qty_kg`, `recommended_qty_kg`, `safety_buffer_pct`, `explanation` JSONB	UQ on `(establishment, dish, date, slot, model_version)` lets you store competing models' predictions for the same target and compare them fairly
I4	`forecast_evaluation`	T3	Accuracy, computed **in SQL** from actuals	`eval_id` PK, `model_version` FK, `establishment_id`, `slot`, `window_start`, `window_end`, `n_predictions`, `mae`, `rmse`, `mape`, `bias`, `computed_at`	Note `bias` (mean signed error), not just MAE. A model that's accurate on average but systematically under-predicts causes real hunger. MAE hides that; bias exposes it

B1.10 GROUP J — Operational Intelligence (2 tables)

#	Table	Tier	Purpose	Key columns	Notes
J1	`alert`	T3	Every actionable signal the system generates	`alert_id` PK, `establishment_id` FK, `alert_type` ('EXPIRY_RISK' / 'WASTE_ANOMALY' / 'CHRONIC_OVERPREP' / 'UNCLAIMED_SURPLUS' / 'NO_REPORTING'), `severity`, `title`, `detail` JSONB, `ref_table`, `ref_id`, `status` ('OPEN' / 'ACK' / 'RESOLVED'), `raised_at`, `acked_by` FK	**The table that turns analytics into a product.** Without it, insights sit in dashboards nobody opens. All five alert types are generated by pure SQL on a schedule — no ML required

#	Table	Tier	Purpose	Key columns	Notes
J2	`waste_cause_cluster`	T3	Vector-clustered root causes	`cluster_id` PK, `label`, `centroid vector(384)`, `member_count`, `dominant_reason_id` FK, `computed_at`	Optional. Assign each `waste_event` its nearest cluster to get emergent cause categories nobody designed upfront

B1.11 MongoDB collections (5)

#	Collection	Purpose	Shape driver	Indexes
M1	`waste_submissions`	Raw field logs, **schema varies by establishment kind**	Restaurant → `dish_waste[]`; household → `bins{}`; college → per-counter; supermarket → SKU array	`{establishment_id:1, service_date:-1}`, `{sync_status:1, submitted_at:1}`, multikey `{"payload.dish_waste.dish_name":1}`, text on `payload.free_text`
M2	`inspection_reports`	Auditor visits: nested checklists, variable sections, photos, free text	Checklist template changes per audit round — a relational schema would need migration every time	`{establishment_id:1, inspected_at:-1}`
M3	`bin_readings`	**Time-series collection** of bin weights	`{timeField:'ts', metaField:'bin_id', granularity:'minutes'}`	native TS index + `{bin_id:1, ts:-1}`
M4	`establishment_meta`	Operating hours per weekday, seasonal notes, arbitrary tags	Every kind has different metadata; ~60% of fields would be NULL relationally	`{establishment_id:1}`
M5	`activity_stream`	Append-only user/UI events (capped collection)	High write, no joins, no updates, TTL-expiring	capped + `{user_id:1, ts:-1}`

The one sentence that justifies MongoDB in your viva: *"A restaurant logs an array of per-dish plate waste, a household logs three bin weights and a photo, a supermarket logs SKU-level markdowns. Relationally that's either 40 nullable columns or an EAV anti-pattern. We store raw in Mongo and project the validated common subset into Postgres."*

B1.12 Vector storage summary

Where	Column	What's embedded	Used for
`food_item`	`name_embedding`	canonical name + description	Resolving messy names during ETL
`recipient_org`	`need_embedding`	free-text need profile	Surplus→recipient matching

Where	Column	What's embedded	Used for
`surplus_listing`	`desc_embedding`	listing description	Surplus→recipient matching
`waste_event`	`notes_embedding`	free-text cause note	Similar-incident retrieval, cause clustering

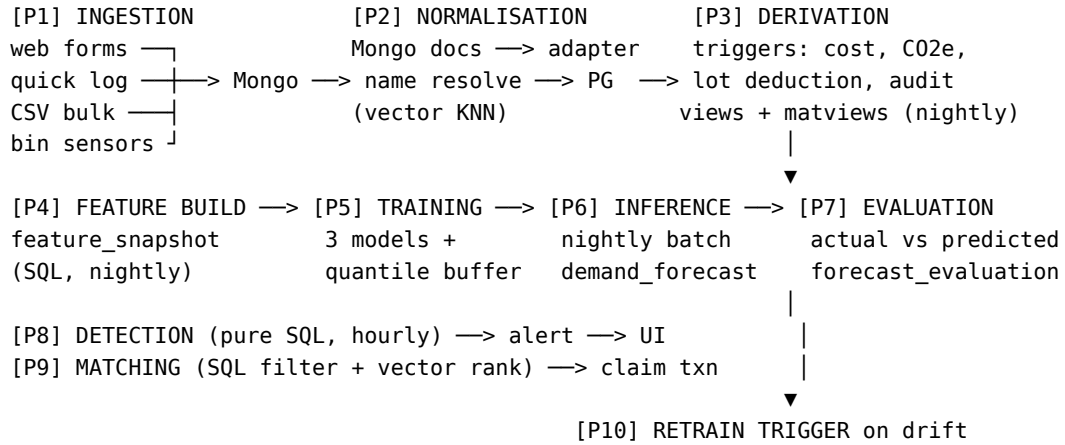
All `vector(384)`, model `all-MiniLM-L6-v2`, cosine, HNSW index. Registered in `embedding_model`.

B1.13 Table count summary

Group	T1	T2	T3	Total
A Geography & Entities	3	3	0	6
B Identity & Audit	1	1	2	4
C Food Reference	2	3	1	6
D Inflow	0	3	0	3
E Preparation & Service	4	1	0	5
F Waste	3	0	0	3
G Redistribution	3	0	0	3
H Context	0	2	0	2
I Forecasting	0	0	4	4
J Ops Intelligence	0	0	2	2
PostgreSQL	**16**	**13**	**9**	**38**
MongoDB collections	1	2	2	5

If you build only the 16 T1 tables, you already have: entities, users, food catalogue, the full prepare→serve→waste loop, and the redistribution marketplace. That is a complete, gradeable project.

B2.0 The full pipeline at a glance



Ten processes. Six run on a schedule, four run on user action. Every one of them either writes to Postgres or reads from it — the database is never bypassed.

B2.1 [P1] INGESTION — four write paths

Path	Who	Destination	Why this destination
Structured web form	Manager / staff at a desk	PostgreSQL, transactional	Data is already well-shaped; needs immediate constraint enforcement
Quick log (mobile-style form)	Kitchen staff mid-shift, household resident	MongoDB `waste_submissions`	Shape differs per establishment kind; speed matters more than structure; validate later
CSV bulk upload	Zone admin importing legacy records	Staging table → validation → PostgreSQL	Needs a dry-run report before commit
Bin reading	Simulated sensor / manual weighing	MongoDB `bin_readings` (time-series)	High frequency, no joins, TTL-expiring

The structured write path in detail (`POST /api/procurement`)

```
zod validation  →  withTx(userId)  →  SET LOCAL app.current_user_id
                →  INSERT procurement (header)
                →  INSERT procurement_line × N
                →  INSERT inventory_lot × N   (expires_at = received_at + category shelf life
                                               if supplier gave no expiry_date)
                →  UPDATE procurement.total_cost = SUM(lines)
                →  COMMIT
```

If line 3 of 5 violates a CHECK, the whole delivery rolls back. The user is not left with a half-recorded invoice. **This is the answer to "why transactions?"** — not "because it was on the requirement list".

Where errors surface

Postgres `RAISE EXCEPTION` messages from triggers are caught in the API layer, mapped to HTTP 422, and rendered in the form as a red inline message. The user sees *"served (48) + leftover (12) exceeds prepared (55) for batch 9021"* — the database's own words. **Show this in the demo.** It proves validation is at the DB level (Req. 12), not just client-side theatre.

B2.2 [P2] NORMALISATION — the Mongo → Postgres ETL bridge

This is the single most interesting process in the system and the best thing to demo. Runs every 10 minutes (cron / route handler) and on-demand from the admin ETL page.

Step-by-step

Step 1 — Claim a batch.

```
db.waste_submissions.find({ sync_status: "PENDING" })
  .sort({ submitted_at: 1 }).limit(200)
// immediately mark as PROCESSING with the run_id to make the job idempotent
```

Step 2 — Route to a shape adapter by `estab_kind`. Five adapters. Each knows one document shape and emits a uniform intermediate record:

```
{ establishment_id, occurred_on, raw_food_name, quantity_kg,
  stage, reason_code, route_name, was_edible, notes, session_ref }
```

- *Commercial adapter:* unwinds `payload.dish_waste[]`; converts `returned_plates × avg_plate_leftover_g / 1000` → kg, stage `POST_CONSUMER_PLATE`; separately emits `buffet_counter_leftovers_kg` as `POST_CONSUMER_SERVICE`; emits `raw_spoilage[]` as `PRE_CONSUMER_SPOILAGE`.
- *Residential adapter:* maps `bins.vegetable_peels_kg` → stage `PRE_CONSUMER_PREP`, `was_edible=false`, reason `PREP_TRIM`; `bins.cooked_leftovers_kg` → `POST_CONSUMER_PLATE`, `was_edible=true`, reason `OVER_PREPARATION`; `bins.expired_packaged_kg` → `PRE_CONSUMER_EXPIRY`.
- *Educational adapter:* per-counter counts × standard portion → kg; reason inferred from turnout gap.
- *Retail adapter:* SKU markdown/expiry rows.
- *Generic fallback:* free-text only → single row with `food_id = NULL`, flagged for review.

Step 3 — Food name resolution (the vector step). A four-stage cascade, cheapest first:

```
1. Exact match on food_item.canonical_name (case-folded)     → confidence 1.00
2. Lookup in food_alias cache                                → cached confidence
3. Trigram similarity > 0.45 on canonical_name               → confidence = similarity
4. Vector KNN: embed raw_name, cosine vs name_embedding      → confidence = 1 - distance
   accept if ≥ 0.72; else food_id = NULL and flag UNRESOLVED
```

Every successful resolution at stages 3–4 writes a `food_alias` row so stage 2 catches it next time. **This is a self-improving cache** — after two weeks of seeded data the vector path fires on maybe 3% of rows, and you can show that decay curve as a chart.

Step 4 — Transactional write to Postgres.

```
BEGIN;
  SET LOCAL app.current_user_id = <submitter>;
  INSERT INTO waste_event (...) VALUES (...) RETURNING waste_id;
  -- trigger tr_waste_enrich fills cost_estimate + co2e_estimate
  -- trigger tr_lot_deduct decrements inventory_lot.remaining_qty if lot_id present
COMMIT;
```

The app **never computes cost or carbon**. The trigger does. That guarantees a report re-run in March reproduces March's numbers even if the app's calculation logic changed.

Step 5 — Close the loop.

```
db.waste_submissions.updateOne({_id}, {$set: {
  sync_status: "SYNCED", sql_waste_ids: [90211, 90212], synced_at: new Date()
}})
```

On failure: `sync_status: "FAILED"` with the Postgres error text, and the admin ETL page lists it with a "retry" button.

Step 6 — Log the run into `etl_run_log`.

Idempotency

`waste_event.source_doc_id` holds the Mongo `_id`. A partial unique index on `(source_doc_id, food_id, stage) WHERE source_doc_id IS NOT NULL` means re-running the ETL on the same document cannot duplicate rows. Say this out loud in the viva — reprocessing safety is the thing most student ETLs get wrong.

B2.3 [P3] DERIVATION — computation that lives in the database

When	Mechanism	What it derives
Instantly, per row	`tr_waste_enrich` (BEFORE INSERT)	`cost_estimate`, `co2e_estimate`
Instantly, per row	`tr_lot_deduct` (AFTER INSERT)	`inventory_lot.remaining_qty`
Instantly, per row	`tr_consumption_balance` (BEFORE I/U)	Rejects impossible quantities
Instantly, per row	`tr_audit`	`audit_log` entries
On demand	`fn_waste_intensity(estab, from, to)`	kg wasted per kg served
On user action	`sp_close_service_session()`	Waste rows + auto surplus listings from leftovers
Every query	Views `v_daily_waste`, `v_preparation_efficiency`, `v_redistribution_performance`	Live rollups
Nightly 02:00	`REFRESH MATERIALIZED VIEW CONCURRENTLY mv_zone_monthly`	Expensive zone×kind×month aggregates

Design rule: if a value can be derived from other columns and must be reproducible, derive it with a trigger and store it. If it's cheap and always current, use a view. If it's expensive and tolerates being a few hours stale, materialise it. Being able to state that rule is worth more than any single feature.

B2.4 [P4] FEATURE BUILD — turning the database into training data

Runs nightly. Populates `feature_snapshot` with one row per `(establishment, dish, target_date, slot)`.

The 18 features, and the SQL that makes each

Feature	Type	Derivation
`lag_1_qty`	numeric	prepared qty for same dish/slot, previous serving day — `LAG(qty,1) OVER (PARTITION BY estab,dish,slot ORDER BY date)`
`lag_7_qty`	numeric	`LAG(qty,7)` — captures weekly rhythm
`lag_14_qty`	numeric	`LAG(qty,14)`
`roll_mean_7`	numeric	`AVG() OVER (... ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING)`
`roll_mean_28`	numeric	same, 28-row frame
`roll_std_28`	numeric	`STDDEV_SAMP() OVER (...)` — volatility drives the safety buffer

Feature	Type	Derivation
`same_dow_mean_4`	numeric	mean of last 4 same-weekday values
`day_of_week`	categorical	`EXTRACT(DOW FROM target_date)`
`is_weekend`	boolean	DOW in (0,6)
`slot`	categorical	from `service_session.slot`
`expected_headcount`	numeric	manager's own entry — the strongest single feature
`headcount_ratio_28`	numeric	expected ÷ 28-day mean actual headcount
`is_event_day`	boolean	EXISTS join through `zone_event` → `locality_event`
`event_impact`	int (-2..2)	from `locality_event.expected_impact`
`days_to_event`	int	signed distance to nearest event
`estab_kind`	categorical	from `establishment_category`
`capacity`	numeric	`establishment.capacity`
`dish_share_28`	numeric	this dish's share of the establishment's 28-day total prepared volume
`actual_qty_kg`	**label**	`SUM(consumption_record.served_qty_kg + leftover... )` — actual *demand*, i.e. served + plate waste, **not** prepared

The label choice is the subtlest decision in the whole ML section. If you train on `prepared_qty_kg` you are teaching the model to reproduce existing over-preparation. Train on **actual consumed demand** (`served_qty_kg + plate_waste_kg`) — what people really ate. Then the recommendation genuinely reduces waste instead of laundering the status quo. Put this paragraph in your report verbatim; it's the kind of reasoning that separates a good project from a competent one.

Leakage guard

Every window frame ends at `1 PRECEDING`. No feature may include the target day's own outcome. State this explicitly — leakage is the first thing a sharp examiner probes.

Split

Time-based only. Months 1–11 train, month 12 validate, months 13–14 test. **Never `train_test_split(shuffle=True)`** on a time series — it lets the model see the future and produces fake 95% accuracy.

B2.5 [P5] TRAINING

Runs manually (or weekly). Python, reads `feature_snapshot` directly from Postgres.

Model	Role	Why it's there
Seasonal naive (`last same-weekday value`)	Baseline	If your ML can't beat this, it has no value. Many student projects never check
Ridge regression	Interpretable linear reference	Coefficients are readable; catches whether the signal is basically linear
LightGBM (L2 objective)	Point forecast	Handles categoricals natively, robust on tabular data, fast
LightGBM (quantile, α=0.85)	Safety buffer	Predicts the 85th percentile of demand directly

Three models plus one quantile head. Not five — nothing in the rubric rewards model count, and every extra model is an hour stolen from the database work you're actually graded on.

Output of training

1. `joblib` artifacts saved to `ml-service/models/lgbm_v3.pkl`
2. A row in `forecast_model` with `algorithm`, `feature_list`, `train_rows`, `val_mae`, `is_active=false`
3. Activation is a deliberate second step: `UPDATE forecast_model SET is_active=false; UPDATE ... SET is_active=true WHERE model_version='lgbm_v3';` — the partial unique index guarantees only one wins.

B2.6 [P6] INFERENCE — how a prediction actually gets made

Nightly batch (03:00)

```
FOR each active establishment
  FOR each dish served in the last 28 days
    FOR each of the next 7 days × applicable slots
      1. Build the feature vector from feature_snapshot (SQL, no Python feature logic)
      2. POST to FastAPI /forecast with the feature row
      3. Receive: predicted_qty_kg (L2 head), q85_qty_kg (quantile head), drivers[]
      4. recommended_qty_kg = q85_qty_kg
         safety_buffer_pct = (q85 - predicted) / predicted × 100
      5. INSERT INTO demand_forecast (..., explanation = drivers JSONB)
```

The recommendation logic, explained properly

A flat "+10% buffer" ignores that volatility differs wildly: a corporate cafeteria on a Tuesday is predictable; a restaurant on a festival Saturday is not. The quantile model learns this per-context. Where `roll_std_28` is low the q85 sits close to the mean (small buffer); where it's high, the gap widens automatically.

State the asymmetry in the report: **under-preparing costs reputation and hunger; over-preparing costs money and waste.** q=0.85 encodes "we accept running short 15% of the time." That number is a policy choice, and it belongs to the zone admin, not the model — so expose it as a configurable field.

What `explanation` JSONB holds

```
{ "baseline_28d_mean": 68.4,
  "same_weekday_mean": 71.2,
  "expected_headcount": 210,
  "headcount_vs_normal": -0.12,
  "event": { "name": "Semester Break", "impact": -2 },
  "top_drivers": [
    {"feature":"expected_headcount","direction":"down","contribution":-7.1},
    {"feature":"roll_mean_7","direction":"up","contribution":+2.3}
  ],
  "volatility_band_kg": [54.0, 71.5] }
```

Rendered in the UI as: *"Prepare **62 kg** (≈ 245 portions). That's 9% below your usual Tuesday because attendance is forecast down 12% during semester break. Range of likely demand: 54–72 kg."*

A number with no reason gets ignored by kitchen staff. The explanation is not a nice-to-have; it's the difference between a feature that's used and one that isn't.

On-demand what-if

The manager changes `expected_headcount` on the forecast page → API rebuilds one feature row → single `/forecast` call → new recommendation, not persisted unless they save it. Fast, interactive, and it demonstrates the model responds to inputs rather than replaying a lookup table.

B2.7 [P7] EVALUATION — closing the loop

Triggered whenever a session is closed (so actuals exist), aggregated nightly into `forecast_evaluation`:

```
INSERT INTO forecast_evaluation (model_version, establishment_id, slot,
       window_start, window_end, n_predictions, mae, rmse, mape, bias, computed_at)
SELECT df.model_version, ss.establishment_id, ss.slot,
       $1, $2, COUNT(*),
       AVG(ABS(a.actual_kg - df.predicted_qty_kg)),
       SQRT(AVG(POWER(a.actual_kg - df.predicted_qty_kg, 2))),
       AVG(ABS(a.actual_kg - df.predicted_qty_kg) / NULLIF(a.actual_kg,0)) * 100,
       AVG(a.actual_kg - df.predicted_qty_kg)                    -- signed bias
FROM demand_forecast df
JOIN service_session ss ON ss.establishment_id = df.establishment_id
     AND ss.service_date = df.target_date AND ss.slot = df.slot
JOIN LATERAL (
  SELECT SUM(cr.served_qty_kg + cr.plate_waste_kg) AS actual_kg
  FROM preparation_batch pb JOIN consumption_record cr USING (batch_id)
  WHERE pb.session_id = ss.session_id AND pb.dish_id = df.dish_id
) a ON TRUE
WHERE ss.is_closed AND ss.service_date BETWEEN $1 AND $2
GROUP BY df.model_version, ss.establishment_id, ss.slot;
```

The accuracy metrics are computed by SQL, not by Python. That is deliberate and it's the reason this belongs in a DBMS project at all: the database is the arbiter of truth about how the model performed.

The business metric that matters more than MAE: waste-per-served-kg on establishments *following* recommendations vs those that aren't. If the system works, that gap is visible. Chart it.

B2.8 [P8] DETECTION — five alerts, all pure SQL, zero ML

Runs hourly, writes into `alert`. This is where analytics becomes a product.

Alert	Logic	Severity
`EXPIRY_RISK`	`inventory_lot` where `remaining_qty > 0 AND expires_at < now() + 24h`, ranked by `remaining_qty × unit_cost`	HIGH if value > ₹2000
`WASTE_ANOMALY`	z-score > 2.5 vs 28-day rolling mean per establishment (window function, §5.8 of main plan)	MEDIUM
`CHRONIC_OVERPREP`	`v_preparation_efficiency` where utilisation < 75% on ≥ 10 of last 20 sessions	HIGH
`UNCLAIMED_SURPLUS`	listings `status='OPEN' AND expires_at < now() + 2h` — uses the partial index	CRITICAL (food is about to be lost *right now*)
`NO_REPORTING`	active establishments with no waste_event and no submission in 5 days	LOW (data-quality, not waste)

Deduplication: unique partial index on `(establishment_id, alert_type) WHERE status = 'OPEN'` — one open alert per type per establishment, no notification storms.

B2.9 [P9] MATCHING — surplus to recipient

Runs on listing creation, and refreshes when a recipient opens their feed.

```
1. SQL hard filter (cheap, index-backed):
     same zone subtree (recursive CTE)
     AND (NOT listing.is_cooked OR recipient.accepts_cooked)
     AND recipient.daily_capacity_kg >= listing.offered_qty_kg * 0.5
     AND listing.expires_at overlaps recipient pickup window
2. Vector rank on the survivors:  1 - (need_embedding <=> desc_embedding)
3. Hybrid score:
     0.60 × cosine_similarity
   + 0.25 × capacity_fit        (1 - |offered - capacity| / capacity, clipped)
   + 0.15 × proximity_score     (haversine, normalised over the zone)
4. Return top 5, store the winning score on the claim
```

Always filter before ranking. Pure vector search would happily recommend 40 kg of chicken biryani to a vegetarian shelter with 5 kg capacity, because “cooked rice dish for hungry people” is semantically perfect. Hard constraints are hard constraints; similarity only orders what’s already feasible. Say this in the demo — it shows you understand what vector search is *for*.

Then the claim transaction (`fn_claim_surplus`, `SELECT ... FOR UPDATE`) from the main plan.

B2.10 [P10] SCHEDULE SUMMARY

Job	Cadence	Duration	Writes to
Mongo→PG ETL	every 10 min	~2 s / 200 docs	`waste_event`, `food_alias`, `etl_run_log`
Embedding backfill (new rows)	every 10 min	~1 s	vector columns
Alert detection	hourly	~1 s	`alert`
Listing expiry sweep	every 15 min	<1 s	`surplus_listing.status = 'EXPIRED'`
Feature snapshot build	nightly 01:00	~30 s	`feature_snapshot`
Matview refresh	nightly 02:00	~10 s	`mv_zone_monthly`
Forecast batch	nightly 03:00	~2 min	`demand_forecast`
Evaluation aggregate	nightly 04:00	~5 s	`forecast_evaluation`
Model retrain	weekly / manual	~3 min	`forecast_model` + artifacts

Implement these as authenticated `POST /api/jobs/<name>` route handlers plus a single `node-cron` runner. That way every job is also a button on the admin page — **essential for demos**, because you cannot wait until 3 AM in front of an examiner.

PART 3: FRONTEND — WHO USES WHAT, AND WHY

B3.0 Five users, five completely different jobs

The mistake most projects make is building one dashboard and bolting role checks onto it. These five people want genuinely different things, on different time horizons, at different speeds.

Persona	Role enum	Time budget per session	Primary verb	Device reality
Kitchen / household logger	`ESTAB_STAFF`	**30 seconds**, mid-shift, greasy hands	*Record*	Phone, one hand, poor light
Establishment manager	`ESTAB_MANAGER`	5 minutes, once or twice a day	*Decide*	Laptop or tablet
Recipient agent (NGO)	`RECIPIENT_AGENT`	2 minutes, urgent, food is expiring	*Claim*	Phone, in a van
Zone administrator	`ZONE_ADMIN`	20 minutes, weekly	*Analyse & intervene*	Desktop, big screen
Auditor	`AUDITOR`	15 minutes, periodic	*Verify*	Desktop, read-only

If the staff logging screen takes more than 30 seconds, **nobody uses it and your entire database is empty.** Design that screen first and hardest. Everything else in the system depends on data that only exists if this one person bothers.

B3.1 PERSONA 1 — Kitchen / household logger (`ESTAB_STAFF`)

Who: a mess cook at 3 PM, a restaurant supervisor at 11 PM, a resident emptying a bin. Not a computer user. Possibly annoyed at being asked.

What they need: to be finished. Fast. With no typing if avoidable.

Screens

Screen	Route	What's on it
Today	`/log`	One card per open session today. Big status: "Lunch — not yet closed". One tap to continue
Quick waste log	`/log/waste`	Big number pad, dish chips (recent-first), stage as 4 large buttons with icons, reason chips, optional voice-to-text note. **No dropdowns.** Submits to Mongo
Close session	`/log/session/[id]/close`	Prepared quantity pre-filled from the batch. Enter actual headcount → served → leftover → plate waste. Live-validating
My submissions	`/log/history`	Last 7 days, sync status badge, tap to correct

The close-session screen in detail — the highest-value screen in the app

```
┌──────────────────────────────────────────┐
│  Lunch · Tue 14 Mar                      │
│  Expected 210  →  Actual  [  187  ]      │
├──────────────────────────────────────────┤
│  Veg Biryani          prepared 62.0 kg   │
│    served   [ 48.5 ]  leftover [ 9.0 ]   │
│    plate waste [ 3.2 ]                   │
│    ✓ 48.5 + 9.0 = 57.5 of 62.0  ok       │
├──────────────────────────────────────────┤
│  Dal Tadka            prepared 28.0 kg   │
│    served   [ 26.0 ]  leftover [ 4.5 ]   │
│    ⚠ 26.0 + 4.5 = 30.5 exceeds 28.0      │
├──────────────────────────────────────────┤
│       [ Close session and submit ]       │
└──────────────────────────────────────────┘
```

Three things happening here worth pointing out in your demo: 1. **Live arithmetic feedback** as they type — mirroring the trigger's rule client-side so the common error is caught before submission. 2. **The trigger still runs.** Client-side maths is UX; the database is truth. Bypass the UI with `curl` in your demo and watch the trigger reject it anyway. 3. Submitting calls `sp_close_service_session`, which auto-creates waste rows *and* auto-lists the 9.0 kg of untouched biryani as surplus. The staff member did not have to think about donation — **the system converted a leftover into a rescue opportunity for free.** That's the moment the project stops looking like a CRUD app.

Components

Numeric stepper with large hit targets · chip selectors · sync-status badge (PENDING/SYNCED/FAILED) · offline-tolerant submit queue (optional but impressive) · inline red error

carrying the Postgres message.

What they cannot do

See analytics, see other establishments, see costs, delete anything. Their nav has three items.

B3.2 PERSONA 2 — Establishment manager (`ESTAB_MANAGER`)

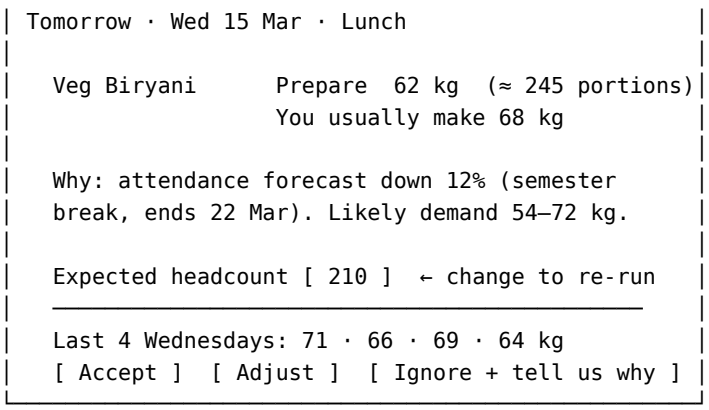
Who: the mess supervisor, restaurant owner, apartment association secretary. Owns one establishment. Cares about money and not looking bad.

What they need: *how much do I cook tomorrow*, *what's about to spoil*, *am I worse than my peers*.

Screens

Screen	Route	Backed by	What it shows
Dashboard	`/e/dashboard`	`v_daily_waste`, `alert`, queries 1, 15, 16	5 KPI cards (waste kg · ₹ lost · kg CO2e · utilisation % · rescue rate), 30-day trend, **open alerts at the top**
Forecast	`/e/forecast`	`demand_forecast`, `/api/forecast`	Next 7 days × dish. "Prepare 62 kg (≈245 portions)" + the explanation panel + editable `expected_headcount` for what-if
Inventory	`/e/inventory`	`inventory_lot`, query 9	Lots sorted by expiry. Colour bands: red < 24 h, amber < 72 h. "Value at risk: ₹4,380". One-click "list as surplus"
Inflow	`/e/procurement`	transactional POST	Multi-line delivery form: add rows, one submit, all-or-nothing
Service planner	`/e/service`	`service_session`, `preparation_batch`	Open a session, set batches (pre-filled with the recommendation), close it
Surplus	`/e/surplus`	matching engine	Active listings, suggested recipients **with visible similarity scores**, claim status
My analytics	`/e/analytics`	queries 4, 5, 7, 14, 17	Waste by stage, by dish, by weekday, cost trend, **peer percentile within my category**

The forecast screen is where the ML earns its place



The **“Ignore + tell us why”** button is worth building. It captures human knowledge the model doesn’t have (“we have a wedding booking”), which is (a) a genuinely good product idea and (b) another free-text field you can embed and cluster.

Peer comparison, handled carefully

Show *“you are in the 78th percentile of College Messes for per-capita waste”* — never *“you are worse than Sunrise Hostel.”* Anonymised peer benchmarking is honest and useful; naming competitors makes the system politically unusable. Mention this design judgement in your report; it’s the kind of thing that reads as maturity.

B3.3 PERSONA 3 — Recipient agent (`RECIPIENT_AGENT`)

Who: an NGO coordinator with a van, an hour before dinner service. Urgency is the entire context.

What they need: what’s available *near me, right now, that I can actually use*, and to claim it before someone else does.

Screens

Screen	Route	Backed by	Notes
Feed	`/r/feed`	matching engine, partial index	Ranked list: match score, quantity, distance, **countdown to expiry**. Filters: veg only, cooked only, max distance, min quantity
Listing detail	`/r/listing/[id]`	`surplus_listing`	Full description, pickup window, establishment contact, “why this matched you”
Claim	modal	`fn_claim_surplus` in a transaction	Partial claim slider (claim 8 of 12 kg). **Optimistic-lock error surfaced honestly:** “Someone claimed this 4 seconds ago — 3 kg still available”

Screen	Route	Backed by	Notes
My claims	`/r/claims`	`redistribution_claim`	Requested / approved / collected. Mark collected on pickup
Our profile	`/r/profile`	`recipient_org`	Edit `need_profile` free text → **re-embeds on save**, changing future matches

The profile screen is a quiet demonstration of the vector layer: edit "we serve 200 vegetarian dinners" to "we now also run a morning breakfast for 80", save, and the feed reorders. Show that live.

The countdown is the design centrepiece

Every listing card shows time-to-expiry as the dominant visual element, red under 60 minutes. This is a system where **the cost of a slow interface is measured in kilograms of food.** Design accordingly.

B3.4 PERSONA 4 — Zone administrator (`ZONE_ADMIN`)

Who: municipal officer, campus sustainability head, or the person who has to write the quarterly report. Sees everything, owns nothing.

What they need: patterns, outliers, and evidence to act on.

Screens

Screen	Route	Backed by	What it shows
Zone overview	`/a/zone`	recursive CTE (query 18), `mv_zone_monthly`	Zone tree with drill-down (Sector → Block → Street), map with sized markers, kind breakdown
Benchmarking	`/a/benchmark`	queries 3, 17	Rank + percentile *within category*, chronic over-preparers, establishments above their category average
Analytics	`/a/analytics`	queries 4, 7, 8, 13, 14	Waste by stage · weekday profile · **event-day vs normal-day** · diversion rate trend · ₹ and CO2e totals
Insights	`/a/insights`	vector KNN, FTS, `waste_cause_cluster`	Free-text search over waste notes · "show me incidents similar to this one" · emergent cause clusters

Screen	Route	Backed by	What it shows
Redistribution	`/a/rescue`	queries 10, 11, 12	Funnel offered→claimed→collected, median pickup time, **and the painful one: surplus that expired unclaimed while recipients had spare capacity in the same zone**
Forecast quality	`/a/models`	`forecast_evaluation`	MAE/RMSE/MAPE/**bias** per model version and slot; active model switcher
Alerts	`/a/alerts`	`alert`	Everything open across the zone, filterable, acknowledge/resolve
Operations	`/a/ops`	`etl_run_log`	ETL runs, failed documents with retry, unresolved food names awaiting confirmation, **manual "run job now" buttons for every scheduled job**
Events	`/a/events`	`locality_event`, `zone_event`	CRUD for festivals/exams/holidays and which zones they hit

The unresolved-names queue is a small screen that impresses

A table of raw names the ETL couldn't confidently resolve, each with the top 3 vector candidates and their scores. One click confirms → writes `food_alias` with `resolution_method='MANUAL'`. **Human-in-the-loop over a vector search**, in about 60 lines of code. Examiners notice this.

Charts worth building (Recharts)

Stacked area (waste by stage over time) · grouped bar (kind comparison) · line with confidence band (forecast vs actual) · horizontal bar (top wasted dishes) · funnel (rescue) · heatmap grid (weekday × slot). Six chart types, each answering a specific question. Don't build a chart you can't state the question for.

B3.5 PERSONA 5 — Auditor (`AUDITOR`)

Who: compliance officer or faculty examiner. **Read-only, zone-wide.**

Why include them: a role with *no write path anywhere* is a clean, provable demonstration of authorisation. Log in as auditor, hit any POST endpoint, get 403 from the API — not from the UI hiding a button.

Screens

Screen	Route	Shows
Compliance	`/x/compliance`	Reporting rate per establishment, segregation compliance, days since last submission
Audit trail	`/x/audit`	`audit_log` — who changed what, before/after JSONB diff, filterable by user/table/date
Data quality	`/x/quality`	Unresolved food names, failed ETL docs, sessions closed without actuals, waste events with NULL food_id
Inspections	`/x/inspections`	MongoDB `inspection_reports`, nested checklists rendered generically

The inspections screen is a neat second Mongo justification: each inspection round uses a different checklist template, so the UI renders whatever nested structure it finds rather than assuming fixed fields.

B3.6 Shared frontend architecture

Navigation

Role determines the entire nav tree at render time in a server component; `middleware.ts` gates the route groups; every API handler re-checks. **Three layers, because hiding a button is not security.**

Data fetching

- Server Components for initial page data (direct `pool.query` — no API round trip)
- Route Handlers for mutations and interactive filters
- No client-side DB access ever (Req. 10)

Reusable components

`KpiCard` · `FilterBar` (date range + zone + kind + stage, URL-synced so filters are shareable) · `DataTable` (server-side sort/filter/paginate — never fetch 450k rows and filter in the browser) · `AlertBanner` · `MatchScoreBadge` · `ExpiryCountdown` · `ExplanationPanel` · `SyncStatusBadge` · `EmptyState`.

Error surfacing

Postgres exception → API catches → maps to 422 with the raw constraint message → rendered inline. **Never swallow a database error into "Something went wrong."** The database's own message is your best evidence for Req. 12.

Empty and loading states

With 450k seeded rows you will rarely see them — which is exactly why they break in the demo when you filter to a date with no data. Build them.

B3.7 Screen count and build order

Persona	Screens	Build in week
Staff logger	4	3
Manager	7	4–6
Recipient	5	6
Zone admin	9	7–8
Auditor	4	8
Auth	2	3
Total	**31**	

Build order, non-negotiable: **auth → staff logger → manager dashboard → recipient feed → admin analytics → auditor.** Each layer generates the data the next one displays. Build the admin dashboard first and you'll spend a week staring at empty charts.

B3.8 The one-paragraph answer if an examiner asks "who is this for?"

"Five roles, three of which write and two of which only read. A kitchen worker logs a meal in under thirty seconds on a phone, and the system converts their leftovers into a surplus listing automatically. A manager gets a quantity recommendation with a stated reason and a confidence range. An NGO sees semantically matched surplus with a countdown. A municipal officer sees which establishments are outliers against their own category, not against the whole zone. An auditor can verify every change through a trigger-written audit log without being able to alter a single row. Each screen exists because a specific person has a specific decision to make, and each is backed by a named query in our catalogue."

PART C — RESEARCH-GRADE EXTENSION (TIER T4)

From "locality food-waste database" to "food-resource intelligence system with a closed evaluation loop"

Parts A and B are a complete, gradeable project on their own. Part C is optional and additive: it does not change a single table, trigger, or route from Parts A/B — it only adds new ones on top, tiered as **T4**, individually cuttable if time runs out (priority order is in §C.8). ## C.0 What this part is and isn't

Parts A and B already give you a strong, gradeable Level-3 DBMS project: 31 Postgres tables + 5 Mongo collections + pgvector, tiered T1/T2/T3, a 10-week plan, a full query catalogue, and a frontend per persona. **None of that is being replaced.**

This part adds one thing on top: a **theory** that turns the schema from "a good CRUD-plus-analytics app" into something closer to a small research system — and a small, precisely scoped set of new tables, pipelines, queries and screens that make that theory concrete and buildable in the time you have left. It also folds in a literature scan of the food-waste/redistribution/shelf-life space so you can state, in the report and in the viva, exactly what gap ZoneFood sits in.

Everything here is additive and tiered as **T4 — Research Extension**, separate from your existing T1–T3. T4 is explicitly optional per-item: cut anything from it without damaging T1–T3, which remains a complete, gradeable project on its own.

C.1 The one-sentence reframe

Old framing: *"Track where food is wasted across a zone and let people claim the surplus."*

New framing:

> ZoneFood treats food as a resource with a **changing state** (not just a quantity), routes that state through an explicit **decision → intervention → measured outcome** loop, and reports how reliable its own data is while doing so.

Everything below exists to make that one sentence literally true in the schema, the pipelines and the dashboards — not just true in the report's introduction.

C.2 Four levels, and where each version of the project sits

```
LEVEL 1 — Ordinary app             "Record what happened."              10 kg waste
LEVEL 2 — Good DBMS (your T1–T3)  "Record it reliably."             + constraints, transactions, audit
LEVEL 3 — Research paper           "Explain/predict one slice of it."    + one model, one metric
LEVEL 4 — ZoneFood v2 (T4, this doc) "Understand state → decide →      Food state → safety/shelf life →
                                   act → measure if it worked."        demand → prevention → surplus →
                                                                    safe redistribution → intervention →
                                                                      measured outcome → better decision
```

Your existing design already reaches deep into Level 2 and touches Level 3 (the forecasting layer). T4 is what closes the loop into Level 4 without inflating scope, because it reuses every table you already have — it does not fork the project into a second system.

C.3 Where this sits in the literature (state this in your report's related-work section)

Four points, each grounded, each stated conservatively (no numbers below are yours until you actually measure them):

1. **The field is fragmented, not empty.** A 2025 systematic review of digitalisation in food-loss-and-waste research (4,277 titles screened, 111 papers included) found that different digital

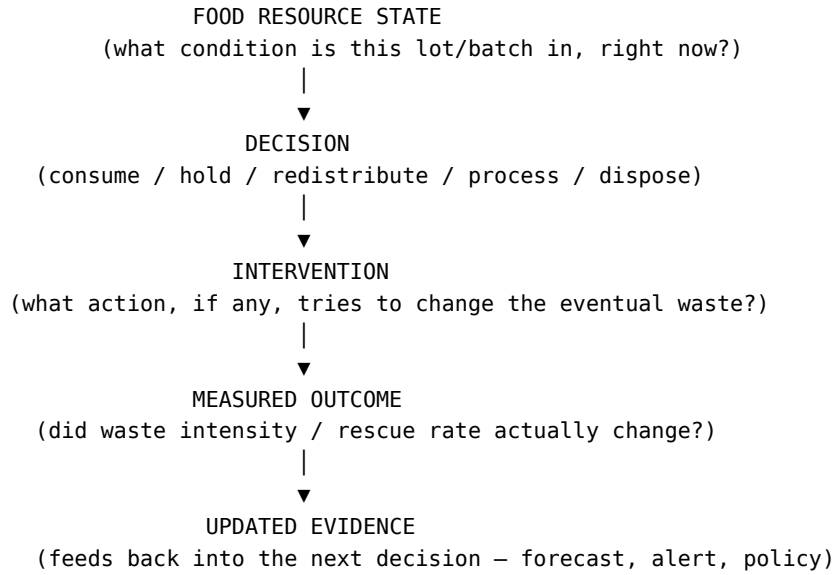
technologies are being applied to isolated parts of the supply chain rather than as unified systems — measurement tools, redistribution platforms, and shelf-life tools are mostly studied separately from one another.

2. **Food-sharing/redistribution platforms are under-evaluated on outcomes, not just under-built.** Reviews of food-waste-reducing digital platforms note that connecting supply and demand digitally is necessary but not sufficient — adoption, trust between organisations, and delivering measurable social/economic/environmental value remain open problems, distinct from the technical matching problem itself.
3. **Shelf-life research is moving from static expiry dates toward dynamic, condition-aware estimation** (temperature/humidity history, multimodal sensing), but this line of work is largely separate from operational food-service databases — it lives in food-science and ML papers, not in the kind of transactional system you're building.
4. **Waste-tracking data itself is measurably unreliable.** A recent study of food-waste tracking in healthcare and hotel kitchens found substantial staff under-reporting (on the order of ~30% overall, worse in some service settings), and found that measurement frequency/coverage was a strong predictor of that bias. Most systems — including your current T1–T3 design — implicitly assume "what staff logged = what happened."

The gap ZoneFood v2 can legitimately claim: not "nobody has built a food-waste app" (many have), but *"there is room for one operational, locality-scale system that keeps the transactional integrity of a real database while treating food state, safety, and its own data reliability as first-class modelled concepts, and that closes the loop by measuring whether its own recommendations changed outcomes."* That is a defensible, specific, checkable claim — put a version of it in your abstract.

Cite these as a short related-work paragraph, not as a wall of citations; you're a DBMS project, not a survey paper. Two to four references is plenty.

C.4 The central theory, formalised as a pipeline



Every new table in §5–§6 exists to answer exactly one of these four questions for one real row in your existing schema. If a proposed table doesn't answer "state / decision / intervention / outcome" for something that already happens in `preparation_batch`, `inventory_lot`, or `waste_event`, it doesn't belong in T4 — cut it. This is the same discipline your Getting Started guide already applies to T1–T3 ("which database concept does this feature demonstrate, and why is that concept the right tool here?"); T4 just asks the equivalent question about *theory*, not mechanism.

C.5 New table group K — Food State, Safety & Shelf Life

Six objects. Attaches to your **existing** `inventory_lot` and `preparation_batch` rows — it does not duplicate them.

#	Table	Attaches to	Purpose
K1	`food_storage_profile`	`food_category` (1:1)	Declared storage requirements and life expectancy, split by raw / prepared / opened
K2	`lot_temperature_summary`	`inventory_lot` (1:1, derived)	Rolled-up temperature history — min/max/mean, excursion count, sourced from Mongo
K3	`food_quality_assessment`	`inventory_lot` or `preparation_batch`	Human-observed freshness/appearance, independent of the expiry clock
K4	`food_safety_assessment`	`inventory_lot` or `preparation_batch`	The regulatory/eligibility gate — this is what actually decides "may this be redistributed"
K5	`food_nutrition`	`food_item` (1:1)	Energy, macros — per 100 g, static reference data
K6	`food_allergen` + `food_item_allergen`	`food_item` (M:N)	Allergen tags — a lookup + bridge, same pattern as your existing `dish_ingredient`

Why K3 and K4 are two tables, not one

This is a deliberate modelling decision and a good thing to defend in the viva: **quality and safety are different axes.** Food can be perfectly *safe* (within its regulatory window, no temperature abuse) but low *quality* (wilted, visually unappealing) — that food should still be eligible for redistribution. Conversely food can look fine but be *unsafe* (undocumented temperature excursion) — that food must never reach a recipient regardless of appearance. Collapsing these into one "grade A/B/C" column, as the earlier draft explicitly warned against, hides exactly the distinction that matters for a redistribution system. Keep them separate; derive a single `redistribution_eligible` boolean from `food_safety_assessment` alone, never from quality.

DDL — `db/migrations/010_food_state_and_safety.sql`

```
CREATE TYPE storage_type      AS ENUM ('AMBIENT','REFRIGERATED','FROZEN');
CREATE TYPE quality_status    AS ENUM ('FRESH','GOOD','ACCEPTABLE','DECLINING','POOR');
CREATE TYPE safety_status     AS ENUM ('COMPLIANT','HOLD','NOT_ELIGIBLE','EXPIRED');
CREATE TYPE assessment_source AS ENUM ('MANUAL','SENSOR_DERIVED','RULE_DERIVED');

CREATE TABLE food_storage_profile (
  food_category_id       INT PRIMARY KEY REFERENCES food_category(food_category_id) ON DELETE CASCADE,
  storage_type            storage_type NOT NULL DEFAULT 'AMBIENT',
  min_temp_c               NUMERIC(4,1),
  max_temp_c               NUMERIC(4,1),
```

```
  max_humidity_pct         NUMERIC(4,1),
  raw_shelf_life_hours      INT CHECK (raw_shelf_life_hours > 0),
  prepared_shelf_life_hours INT CHECK (prepared_shelf_life_hours > 0),
  opened_shelf_life_hours   INT CHECK (opened_shelf_life_hours > 0),
  notes                     TEXT,
  CONSTRAINT ck_temp_range CHECK (max_temp_c IS NULL OR min_temp_c IS NULL OR max_temp_c >= min_temp_c)
);

-- Derived nightly from the Mongo food_temperature_readings collection (see §7).
CREATE TABLE lot_temperature_summary (
  lot_id            BIGINT PRIMARY KEY REFERENCES inventory_lot(lot_id) ON DELETE CASCADE,
  min_temp_c        NUMERIC(4,1),
  max_temp_c        NUMERIC(4,1),
  mean_temp_c       NUMERIC(4,1),
  excursion_count   INT NOT NULL DEFAULT 0 CHECK (excursion_count >= 0),
  excursion_minutes INT NOT NULL DEFAULT 0 CHECK (excursion_minutes >= 0),
  last_excursion_at TIMESTAMPTZ,
  reading_count     INT NOT NULL DEFAULT 0,
  computed_at       TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE food_quality_assessment (
  assessment_id     BIGSERIAL PRIMARY KEY,
  lot_id            BIGINT REFERENCES inventory_lot(lot_id) ON DELETE CASCADE,
  batch_id          BIGINT REFERENCES preparation_batch(batch_id) ON DELETE CASCADE,
  quality_status    quality_status NOT NULL,
  appearance_notes  TEXT,
  packaging_intact  BOOLEAN,
  assessed_by       INT REFERENCES app_user(user_id),
  assessed_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT ck_quality_target CHECK (
    (lot_id IS NOT NULL AND batch_id IS NULL) OR (lot_id IS NULL AND batch_id IS NOT NULL)
  )
);

CREATE TABLE food_safety_assessment (
  safety_id              BIGSERIAL PRIMARY KEY,
  lot_id                 BIGINT REFERENCES inventory_lot(lot_id) ON DELETE CASCADE,
  batch_id                BIGINT REFERENCES preparation_batch(batch_id) ON DELETE CASCADE,
  safety_status           safety_status NOT NULL,
  source                  assessment_source NOT NULL DEFAULT 'RULE_DERIVED',
  redistribution_eligible BOOLEAN NOT NULL DEFAULT FALSE,
  hold_reason             TEXT,
  regulatory_ref          VARCHAR(60),          -- e.g. FSSAI surplus-food guideline clause
  assessed_by             INT REFERENCES app_user(user_id),
  assessed_at             TIMESTAMPTZ NOT NULL DEFAULT now(),
  released_by             INT REFERENCES app_user(user_id),
  released_at             TIMESTAMPTZ,
  CONSTRAINT ck_safety_target CHECK (
    (lot_id IS NOT NULL AND batch_id IS NULL) OR (lot_id IS NULL AND batch_id IS NOT NULL)
  ),
  CONSTRAINT ck_hold_needs_reason CHECK (safety_status <> 'HOLD' OR hold_reason IS NOT NULL),
  CONSTRAINT ck_eligible_needs_compliant CHECK (
    NOT redistribution_eligible OR safety_status = 'COMPLIANT'
  )
```

```
);

CREATE TABLE food_nutrition (
  food_id              INT PRIMARY KEY REFERENCES food_item(food_id) ON DELETE CASCADE,
  energy_kcal_per_100g NUMERIC(6,1) CHECK (energy_kcal_per_100g >= 0),
  protein_g_per_100g   NUMERIC(5,2) CHECK (protein_g_per_100g >= 0),
  carbs_g_per_100g     NUMERIC(5,2) CHECK (carbs_g_per_100g >= 0),
  fat_g_per_100g       NUMERIC(5,2) CHECK (fat_g_per_100g >= 0),
  fiber_g_per_100g     NUMERIC(5,2) CHECK (fiber_g_per_100g >= 0),
  sodium_mg_per_100g   NUMERIC(6,1) CHECK (sodium_mg_per_100g >= 0),
  source               VARCHAR(120)             -- e.g. IFCT 2017 / manufacturer label
);

CREATE TABLE food_allergen (
  allergen_id   SERIAL PRIMARY KEY,
  allergen_name VARCHAR(40) NOT NULL UNIQUE       -- Peanuts, Tree Nuts, Dairy, Gluten, Soy, Egg, Shellfish
);

CREATE TABLE food_item_allergen (
  food_id     INT NOT NULL REFERENCES food_item(food_id) ON DELETE CASCADE,
  allergen_id INT NOT NULL REFERENCES food_allergen(allergen_id) ON DELETE CASCADE,
  PRIMARY KEY (food_id, allergen_id)
);
```

Key rule that must live in a trigger, not just app code: `redistribution_eligible` on `food_safety_assessment` can never be `TRUE` unless `safety_status = 'COMPLIANT'` — this is already expressed as a CHECK above, but the *value* of `safety_status` itself should be computed by a `tr_derive_safety_status` trigger from `lot_temperature_summary.excursion_count`, `inventory_lot.expires_at` vs `now()`, and any open `HOLD`. That gives you a genuine trigger-derived-field demo that's more interesting than the ones in T1–T3, because it fans in from three source tables instead of one.

C.6 New table group L — Traceability, Intervention & Data Quality

Four objects, plus one small extension to a table you already have.

#	Table	Purpose
L1	`batch_lot_consumption`	The missing bridge: which raw lots fed which prepared batch. Closes the supplier→lot→batch→waste chain.
L2	`waste_prevention_action`	A proposed or taken intervention, tied to a cause.
L3	`intervention_outcome`	Before/after waste intensity for one action — the evaluation record.
L4	`data_quality_measurement`	Per establishment, per period: how much of the expected data actually arrived, and how.

6.1 Why `batch_lot_consumption` is the single highest-value addition in this whole document

Look at what you already have: `inventory_lot.source_line_id → procurement_line → procurement.supplier_id`, and `waste_event.lot_id` / `waste_event.session_id`, and `preparation_batch.session_id`. The **only missing link** is which lot(s) fed which prepared batch — right now `preparation_batch` only points at `dish`, and `dish_ingredient` only says which *food items* a dish needs in principle, not which *physical lot* was drawn down on a given day. Add that one bridge table and you get, for free, the traceability chain the earlier discussion asked for:

```
supplier → procurement_line → inventory_lot → batch_lot_consumption → preparation_batch
                                                                              │
                                                                              ▼
                                                                    session → waste_event
```

Now "this supplier's tomatoes are associated with spoilage-tagged waste at three different establishments this month" becomes a five-table JOIN, not a hypothetical. This is the query that makes an examiner sit up — it's a genuinely non-obvious insight that only exists because the schema supports it.

DDL — `db/migrations/011_traceability_and_intervention.sql`

```sql
CREATE TYPE action_status AS ENUM ('PROPOSED','ACCEPTED','REJECTED','IMPLEMENTED');
CREATE TYPE action_type   AS ENUM (
  'REDUCE_PREP_QTY','ADJUST_STORAGE','EARLY_SURPLUS_LISTING',
  'EXPAND_MATCH_RADIUS','PORTION_SIZE_CHANGE','COLD_CHAIN_ALERT','OTHER'
);

CREATE TABLE batch_lot_consumption (
  batch_id       BIGINT NOT NULL REFERENCES preparation_batch(batch_id) ON DELETE CASCADE,
  lot_id         BIGINT NOT NULL REFERENCES inventory_lot(lot_id) ON DELETE RESTRICT,
  qty_consumed_kg NUMERIC(10,3) NOT NULL CHECK (qty_consumed_kg > 0),
  PRIMARY KEY (batch_id, lot_id)
);
-- Enforced by a trigger, not a CHECK (needs a cross-row SUM):
--   SUM(qty_consumed_kg) for a lot across all batches <= inventory_lot.received_qty

CREATE TABLE waste_prevention_action (
  action_id         BIGSERIAL PRIMARY KEY,
  establishment_id  INT NOT NULL REFERENCES establishment(establishment_id) ON DELETE CASCADE,
  reason_id         INT REFERENCES waste_reason(reason_id),
  action_type       action_type NOT NULL,
  description       TEXT NOT NULL,
  recommended_by    VARCHAR(20) NOT NULL DEFAULT 'SYSTEM' CHECK (recommended_by IN ('SYSTEM','MANAGER','ADMIN')),
  status            action_status NOT NULL DEFAULT 'PROPOSED',
  target_dish_id    INT REFERENCES dish(dish_id),
  proposed_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  decided_by        INT REFERENCES app_user(user_id),
  decided_at        TIMESTAMPTZ
);

CREATE TABLE intervention_outcome (
  outcome_id              BIGSERIAL PRIMARY KEY,
  action_id               BIGINT NOT NULL REFERENCES waste_prevention_action(action_id) ON DELETE CASCADE,
  baseline_window_start   DATE NOT NULL,
  baseline_window_end     DATE NOT NULL,
  followup_window_start   DATE NOT NULL,
```

```
  followup_window_end      DATE NOT NULL,
  baseline_waste_intensity NUMERIC(8,4),   -- kg waste / meal served, computed by SQL, not by the app
  followup_waste_intensity NUMERIC(8,4),
  pct_change               NUMERIC(6,2) GENERATED ALWAYS AS (
    CASE WHEN baseline_waste_intensity IS NULL OR baseline_waste_intensity = 0 THEN NULL
         ELSE ROUND(100.0 * (followup_waste_intensity - baseline_waste_intensity) / baseline_waste_intensity, 2)
    END
  ) STORED,
  est_kg_rescued     NUMERIC(10,3),
  est_cost_saved     NUMERIC(10,2),
  computed_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT ck_windows CHECK (baseline_window_end < followup_window_start)
);

CREATE TABLE data_quality_measurement (
  measurement_id      BIGSERIAL PRIMARY KEY,
  establishment_id    INT NOT NULL REFERENCES establishment(establishment_id) ON DELETE CASCADE,
  period_start         DATE NOT NULL,
  period_end           DATE NOT NULL,
  expected_sessions    INT NOT NULL CHECK (expected_sessions >= 0),
  logged_sessions      INT NOT NULL CHECK (logged_sessions >= 0),
  coverage_pct         NUMERIC(5,2) GENERATED ALWAYS AS (
    CASE WHEN expected_sessions = 0 THEN NULL
         ELSE ROUND(100.0 * logged_sessions / expected_sessions, 2)
    END
  ) STORED,
  manual_pct           NUMERIC(5,2),   -- % of that period's waste_event rows with source = 'MANUAL_SCALE'
  estimated_pct        NUMERIC(5,2),   -- % with source = 'ESTIMATED'
  computed_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_dq_period UNIQUE (establishment_id, period_start, period_end)
);
```

6.2 The ALTER + DROP migration you need anyway — make it earn its keep

Your Getting Started guide is explicit that the project must demonstrate at least one `ALTER` and one genuinely-unused `DROP`, and notes most teams forget it. T4 gives you a real, motivated one instead of an artificial one:

```
-- db/migrations/012_waste_event_measurement_quality.sql
ALTER TABLE waste_event
  ADD COLUMN measurement_source assessment_source NOT NULL DEFAULT 'MANUAL',
  ADD COLUMN measurement_confidence NUMERIC(3,2) CHECK (measurement_confidence BETWEEN 0 AND 1);

-- Later, once you confirm nobody populated it (a genuinely unused column, not a staged one):
ALTER TABLE waste_event DROP COLUMN IF EXISTS legacy_unit_note;
```

This satisfies the requirement *and* is the column that `data_quality_measurement.manual_pct` / `estimated_pct` actually aggregate from — it is not busywork, it's load-bearing.

C.7 MongoDB extension — one new collection

Collection	Purpose	Shape	Indexes
`food_temperature_readings`	High-frequency temperature readings per lot, source can be sensor or manual spot-check	Time-series: `{timeField:'ts', metaField:'lot_id', granularity:'minutes'}`	native TS index + `{lot_id:1, ts:-1}`

This follows the exact same justification your existing `bin_readings` (M3) collection already uses — high-write, append-only, no joins, time-series-shaped data belongs in Mongo's native time-series collection type, not in Postgres. A nightly (or on-demand) job aggregates each lot's readings into the single `lot_temperature_summary` row in Postgres described in §5 — same **document store at the edge, relational at the core** pattern you already use for `waste_submissions`.

```
db.createCollection("food_temperature_readings", {
  timeseries: { timeField: "ts", metaField: "lot_id", granularity: "minutes" }
});
db.food_temperature_readings.insertOne({
  ts: new Date(), lot_id: 4821, temp_c: 6.2, source: "SENSOR", device_id: "FRIDGE-03"
});
```

If you don't have real sensors, seed this synthetically the same way your existing seed generator already fabricates realistic monsoon/seasonal effects — that's an accepted, stated simplification, not a weakness, as long as you say so in the report.

C.8 Updated tier system

Tier	Objects	Meaning	Build in
T1 — Core	16	No product without these	Weeks 1–4 (unchanged)
T2 — Extended	13	"Beyond CRUD"	Weeks 4–7 (unchanged)
T3 — Intelligence	9–10	Forecasting, alerts, ops	Weeks 7–9 (unchanged)
T4 — Research Extension	**10** (8 Postgres tables incl. bridge/ lookup, 1 Mongo collection, 1 ALTER set)	Food state, safety, traceability, intervention evaluation, data quality	**Weeks 9–10, cuttable item-by-item**

T4 sub-priority — cut from the bottom up if time is short:

1. **Essential** (cheap, high payoff, do these even under time pressure): `batch_lot_consumption`, the `waste_event` ALTER, `food_safety_assessment` + `food_quality_assessment`.
2. **Strong** (do if Week 9 exists): `food_storage_profile`, `lot_temperature_summary` + the Mongo collection, `data_quality_measurement`.
3. **Stretch** (only if everything else is frozen and tested): `waste_prevention_action` + `intervention_outcome` (the full closed-loop demo), `food_nutrition`/`food_allergen`.

This mirrors — deliberately — the same essential/strong/stretch framing already used for the T1/T2/T3 split, so grading logic stays consistent across the whole document set.

C.9 New processing pipelines

These slot into the pipeline numbering already established in the companion Tables/Processing document (P1–P10) as P11–P13.

[P11] STATE DERIVATION — nightly, pure SQL + one trigger

For every open `inventory_lot` and undischarged `preparation_batch`: 1. Pull `lot_temperature_summary` (already aggregated from Mongo). 2. Compare `now()` against `inventory_lot.expires_at` and `food_storage_profile.*_shelf_life_hours`. 3. Trigger `tr_derive_safety_status` sets `food_safety_assessment.safety_status`: - `EXPIRED` if past the regulatory expiry, - `HOLD` if `excursion_count > 0` and no human release yet, - `NOT_ELIGIBLE` if flagged unsafe by a manual `food_quality_assessment` of `POOR`, - else `COMPLIANT`. 4. `redistribution_eligible` flips to `TRUE` only when `safety_status = 'COMPLIANT'` **and** remaining shelf life exceeds a configurable minimum pickup window (e.g. 90 minutes) — this is the rule that should feed `surplus_listing` creation, tying T4 directly back into your existing T1 redistribution flow instead of sitting beside it.

[P12] INTERVENTION EVALUATION — the loop-closer

Triggered when a `waste_prevention_action.status` moves to `IMPLEMENTED`: 1. Compute `baseline_waste_intensity` as `SUM(waste_event.quantity_kg) / SUM(service_session.actual_headcount)` over the `baseline_window` (e.g. the 2 weeks before the action). 2. Wait for `followup_window` to elapse (e.g. 2 weeks after). 3. Compute `followup_waste_intensity` the same way. 4. Insert one row into `intervention_outcome`; `pct_change` is a generated column — no drift between what's stored and what's computed. 5. Surface this on the manager and admin dashboards (see §11).

This is deliberately simple — a pre/post comparison, not a randomised controlled trial. **Say this explicitly in the report**: "we measure association between an intervention and a subsequent change in waste intensity for the same establishment; this is not a controlled experiment and does not establish causation." That sentence alone will read as more scientifically mature than an unqualified "our system reduces waste by X%" claim, and it directly answers the concern about over-claiming impact that the literature review raised.

[P13] DATA QUALITY SCORING — weekly, pure SQL

For each establishment, each week:

```
INSERT INTO data_quality_measurement
  (establishment_id, period_start, period_end, expected_sessions, logged_sessions, manual_pct, estimated_pct)
SELECT
  e.establishment_id,
  date_trunc('week', now())::date - 7,
  date_trunc('week', now())::date - 1,
  -- expected: one row per (date, slot) the establishment is configured to serve
  (SELECT COUNT(*) FROM generate_series(..)) AS expected_sessions,
  COUNT(DISTINCT ss.session_id) AS logged_sessions,
  ROUND(100.0 * COUNT(*) FILTER (WHERE we.measurement_source = 'MANUAL')   / NULLIF(COUNT(*),0), 2),
  ROUND(100.0 * COUNT(*) FILTER (WHERE we.measurement_source = 'ESTIMATED') / NULLIF(COUNT(*),0), 2)
FROM establishment e
LEFT JOIN service_session ss ON ss.establishment_id = e.establishment_id AND ss.service_date >= now() - interval
LEFT JOIN waste_event we ON we.session_id = ss.session_id
GROUP BY e.establishment_id;
```

This single query is your evidence that ZoneFood v2 does not silently trust its own inputs — a direct, checkable answer to the under-reporting problem raised in §3.4.

C.10 New/extended query catalogue (add to your existing Req. 6 catalogue)

Twelve queries, each demonstrating a specific SQL feature **and** answering a T4 research question. Numbered to continue after your existing catalogue.

#	Question	Demonstrates
Q23	Which food categories accumulate the most temperature excursion minutes, and does that correlate with their spoilage-tagged waste share?	JOIN + GROUP BY + correlation via two aggregates in one query
Q24	For each supplier, what fraction of the lots they supplied eventually appear in a spoilage-tagged `waste_event`, traced through `batch_lot_consumption`?	5-table JOIN, the traceability chain from §6.1
Q25	List all currently `HOLD` lots with no release action in the last 4 hours (an operational backlog view).	Subquery / NOT EXISTS, time-windowed filter
Q26	Rank establishments by `pct_change` in `intervention_outcome`, restricted to actions of type `REDUCE_PREP_QTY`.	Window function `RANK() OVER (...)`, filtered aggregate
Q27	For every `WASTE_EVENT`, what is the eligible-vs-actual redistribution gap — i.e. `redistribution_eligible = TRUE` lots/batches whose food never appears in a `surplus_listing`?	LEFT JOIN + IS NULL anti-join, directly answers "unclaimed rescuable surplus"
Q28	Compare `mae`/`bias` in `forecast_evaluation` for establishments that *did* vs *did not* act on `waste_prevention_action` recommendations in the same period.	CASE-based cohort split + aggregate comparison — validates the forecast layer against real behaviour, not just against held-out data
Q29	Nutrition-weighted redistribution: total protein (g) rescued via `redistribution_claim`, by recipient org, last 30 days.	JOIN across `redistribution_claim → surplus_listing → food_item → food_nutrition`, weighted SUM

#	Question	Demonstrates
Q30	Data-quality-adjusted waste ranking: rank establishments by waste intensity **only** among those with `coverage_pct >= 80` in `data_quality_measurement` — show how the ranking changes vs. the unadjusted version.	Demonstrates why data-quality gating matters — produces two different leaderboards from the same base data
Q31	Allergen-safe matching audit: for each `redistribution_claim`, confirm the claimed food's allergens do not intersect the recipient's declared restrictions (if you extend `recipient_org` with a `restricted_allergens` array).	Array/set operations, a genuine hard-constraint check
Q32	Root-cause propagation: supplier lots implicated in spoilage waste at **more than one** establishment in the same week.	`HAVING COUNT(DISTINCT establishment_id) > 1` — the "this isn't three unrelated problems" query
Q33	Shelf-life honesty check: for lots marked `EXPIRED`, how much of their `remaining_qty` was wasted vs. redistributed before expiry — did the system act in time?	CASE aggregation over lifecycle outcome
Q34	Recursive zone rollup of `intervention_outcome.est_kg_rescued`, using your existing zone recursive CTE.	Reuses your existing recursive CTE machinery — cheap to add, ties T4 into T1 geography

C.11 Frontend additions (per persona, additive to your existing screen list)

Persona	New screen/element	What it shows
Staff (`ESTAB_STAFF`)	Quality/safety quick-tag on close-session screen	3-tap `quality_status` + `safety_status` entry per batch — no new navigation, folded into the existing highest-value screen

Persona	New screen/element	What it shows
Manager (ESTAB_MANAGER)	“Interventions” tab	List of waste_prevention_action with status chips; accept/reject; once implemented, shows the intervention_outcome before/after bars when ready
Recipient agent (RECIPIENT_AGENT)	Safety badge on listing card	COMPLIANT / HOLD badge pulled from food_safety_assessment; HOLD listings simply never render — enforced by the query, not by UI discipline
Zone admin (ZONE_ADMIN)	“Data reliability” panel	coverage_pct, manual_pct, estimated_pct per establishment from data_quality_measurement — this is the panel that makes the whole dashboard credible instead of naively trusting 100% of its own numbers
Zone admin	“Root cause” view	Q32’s output rendered as a small table — supplier lots implicated at >1 establishment
Auditor (AUDITOR)	Traceability drill-down	Given a waste_event, walk backward through batch_lot_consumption → inventory_lot → procurement_line → supplier in one read-only screen

None of these need new route groups — they’re tabs/panels inside screens your existing plan already scopes for Week 6 (dashboards) and Week 8 (authority analytics), so they don’t extend your screen count meaningfully.

C.12 Five-layer evaluation framework (for the report’s “Evaluation” section)

Replace a single “the application was successfully developed” closing statement with five layers, each with a named metric you actually produce from the system:

Layer	Question	Evidence you already have or now have
1. Database correctness	Does the schema refuse impossible states?	Your existing negative-test table, now extended with T4 constraints (e.g. inserting a `redistribution_eligible = TRUE` row with `safety_status = 'HOLD'` must fail)
2. Performance	Does it scale to zone size?	Your existing EXPLAIN ANALYZE before/after index and partition evidence, unchanged
3. Model quality	Is the forecast any good?	Your existing MAE/RMSE/MAPE/bias from `forecast_evaluation`, unchanged
4. Operational quality	Does the system's own machinery work end-to-end?	New: `coverage_pct` from `data_quality_measurement`; match-to-collection rate from `redistribution_claim.match_score` vs `collected_at IS NOT NULL`
5. Outcome / impact	Did anything the system recommended actually change anything?	New: `intervention_outcome.pct_change`, reported as "associated with", not "caused"

State the result of layer 5 carefully — a small pilot dataset with one or two interventions is a legitimate result for a course project ("in our seeded evaluation, establishments that accepted a `REDUCE_PREP_QTY` recommendation showed a median waste-intensity reduction of X% over the following two weeks"), and is a far stronger closing paragraph than an unqualified developed/deployed statement.

C.13 Revised phrasing for your model/forecasting narrative

Don't present the forecast as the contribution. Reframe one sentence in your report:

- Weak: *"We built a food-waste prediction model."*
- Strong: *"Demand forecasting is used only as one input to `waste_prevention_action` recommendations; its value is judged not by MAE alone but by whether establishments that acted on its recommendations show a measurable change in `intervention_outcome`."*

This single reframing is what Q28 above exists to support with an actual query result, not just a claim.

C.14 Scope guardrails — what NOT to add

Everything below was explicitly considered and rejected, for the same reason your Getting Started guide already warns against scope creep toward maps/IoT/mobile/chatbots:

- No microbiology or lab-grade food-safety determination — `food_safety_assessment` is a rules/regulatory gate plus human confirmation, never a claim of biological safety.

- No real IoT hardware — `food_temperature_readings` is seeded synthetically, stated as such.
- No machine-learned dynamic shelf-life model — `food_storage_profile` values are declared reference data (from packaging/FSSAI guidance), not predicted. Dynamic ML-based shelf-life estimation is named in §3 as future work, not built.
- No causal-inference framework for intervention evaluation — plain pre/post comparison, explicitly labelled as association, not causation.
- No blockchain-style traceability — `batch_lot_consumption` is one ordinary bridge table; "traceability" here means a JOIN path, not a distributed ledger.
- No mobile app, computer vision, or drone/hardware components of any kind.

If asked in a viva "why didn't you build X", the answer for every item above is the same sentence pattern: *"X exists in the literature as a separate, specialised research problem; ZoneFood's contribution is the operational integration around it, not X itself."*

C.15 Where T4 fits in your existing 10-week plan

No new weeks — T4 occupies the slack that already exists in Weeks 9–10 (feature freeze / testing / report week) in your original plan, plus anything Week 8 finishes early. Suggested placement:

Week	Addition
8 (tail end, if T3 finishes on schedule)	`batch_lot_consumption`, the `waste_event` ALTER, `food_safety_assessment` + `food_quality_assessment`, P11
9	`food_storage_profile`, Mongo `food_temperature_readings`, `lot_temperature_summary`, `data_quality_measurement`, P13, Q23–Q25, Q30, Q32
9–10 (only if ahead of schedule)	`waste_prevention_action`, `intervention_outcome`, P12, Q26–Q29, Q31, Q33–Q34, the report/frontend additions in §11

If Week 9 arrives and T1–T3 testing isn't fully done: **stop here and don't start T4 at all.** A complete, well-tested T1–T3 project outscores a T1–T3 project with test gaps plus a half-built T4 every time. T4's entire value is that it's optional — treat that as load-bearing, not as a throwaway disclaimer.

C.16 One-paragraph project definition, for the abstract / viva opener

> ZoneFood is a locality-scale, multi-paradigm database system that tracks food across heterogeneous urban establishments not merely as a quantity moving through procurement, preparation and waste, but as a resource with a state — safety, quality and remaining usable life — that determines whether it can be safely redistributed. The system combines PostgreSQL (transactional core and invariants), MongoDB (heterogeneous field logs and time-series sensor data) and pgvector (semantic matching and name resolution) to move food from a decision point through an optional intervention to a measured outcome, and it explicitly reports the reliability of its own underlying data rather than assuming it. The core database engineering — normalisation, constraints, concurrency-safe transactions, indexing, partitioning and materialised views — remains the primary deliverable; the

food-state and intervention-evaluation layer gives that engineering a coherent reason to exist beyond CRUD.

C.17 Summary of every new object introduced in this document

Object	Type	Tier	One-line purpose
`food_storage_profile`	Postgres table	T4-strong	Declared storage/ shelf-life reference data per food category
`lot_temperature_summary`	Postgres table	T4-strong	Rolled-up temperature history per lot, derived from Mongo
`food_quality_assessment`	Postgres table	T4-essential	Human freshness/ appearance observation
`food_safety_assessment`	Postgres table	T4-essential	The regulatory eligibility gate for redistribution
`food_nutrition`	Postgres table	T4-stretch	Static macro reference data per food item
`food_allergen` / `food_item_ allergen`	Postgres lookup + bridge	T4-stretch	Allergen tagging
`batch_lot_consumption`	Postgres bridge	T4-essential	Closes the sup- plier→lot→batch→waste traceability chain
`waste_prevention_action`	Postgres table	T4-stretch	A proposed/taken intervention
`intervention_outcome`	Postgres table	T4-stretch	Pre/post measurement of one intervention
`data_quality_measurement`	Postgres table	T4-strong	Coverage and measurement-method reliability per establishment/week
`food_temperature_readings`	Mongo time-series collection	T4-strong	Raw high-frequency lot temperature readings
`waste_event.measurement_source` / `.measurement_confidence`	ALTER on existing table	T4-essential	Satisfies the required ALTER demo *and* feeds `data_quality_measurement`

10 new storage objects (8 Postgres + 1 Mongo + 1 motivated ALTER/DROP pair), 3 new pipelines, 12 new queries, 6 new/extended screens, a 5-layer evaluation framework, and one literature-grounded gap statement — built entirely on top of your existing 39-object T1–T3 design without altering any of it.

PART D — CONSOLIDATED REFERENCE

D1. Every storage object, every tier, one table

Group	T1	T2	T3	T4	Total
A — Geography & Entities	3	3	0	0	6
B — Identity & Audit	1	1	2	0	4
C — Food Reference	2	3	1	0	6
D — Food Inflow	0	3	0	0	3
E — Preparation & Service	4	1	0	0	5
F — Waste	3	0	0	0	3
G — Redistribution	3	0	0	0	3
H — Context	0	2	0	0	2
I — Forecasting & Evaluation	0	0	4	0	4
J — Operational Intelligence	0	0	2	0	2
K — Food State, Safety & Shelf Life	0	0	0	6	6
L — Traceability, Intervention & Data Quality	0	0	0	4	4
PostgreSQL subtotal	**16**	**13**	**9**	**10**	**48**
MongoDB collections	1	2	2	1	6
Grand total (Postgres + Mongo)	**17**	**15**	**11**	**11**	**54**

(Counting `food_item_allergen` as part of the K6 pair and `lot_temperature_summary`'s Mongo source separately from its Postgres summary table; see Part C §C.17 for the itemised T4 list.)

Build discipline, unchanged by adding T4: T1 alone is a demoable system. T1+T2+T3 is a complete, gradeable Level-3 project — this is still true with zero T4 objects built. T4 is evidence of depth beyond the rubric's minimum, not a substitute for finishing T1–T3 properly. If Week 9 arrives and T1–T3 testing isn't done, stop and do not start T4.

D2. Requirements-to-deliverable map (unified)

The full requirement-by-requirement mapping is in Part A §14. T4 strengthens, but does not change the mapping for, the following requirements specifically:

Req.	How T4 strengthens it
Req. 5 (DDL: CREATE/ALTER/DROP)	The `waste_event` ALTER in §C.6.2 is a motivated, load-bearing example rather than a token one

Req.	How T4 strengthens it
Req. 6 (DML depth)	Twelve further queries (Q23–Q34, §C.10), including a five-table traceability JOIN and a generated-column-based pre/post comparison
Req. 7 (advanced concepts)	A second, three-source trigger (`tr_derive_safety_status`) and a second Mongo time-series collection
Req. 14 (documentation, testing)	A fifth evaluation layer (§C.12) beyond correctness/performance/model-quality
Req. 17/18 (originality, beyond CRUD)	The state→decision→intervention→outcome loop and the literature-grounded gap statement (§C.1–C.3) are the strongest originality evidence in the whole project

D3. The project, in one paragraph, at each depth

If you built only Part A (T1–T3): *"ZoneFood is a locality-scale, multi-paradigm database system that tracks food from procurement through preparation, service, waste and redistribution across heterogeneous urban establishments, using PostgreSQL for transactional integrity, MongoDB for heterogeneous field logs, and pgvector for semantic matching and name resolution."*

If you also built Part C (T4): *"ZoneFood is a locality-scale, multi-paradigm database system that tracks food across heterogeneous urban establishments not merely as a quantity moving through procurement, preparation and waste, but as a resource with a state — safety, quality and remaining usable life — that determines whether it can be safely redistributed. The system combines PostgreSQL (transactional core and invariants), MongoDB (heterogeneous field logs and time-series sensor data) and pgvector (semantic matching and name resolution) to move food from a decision point through an optional intervention to a measured outcome, and it explicitly reports the reliability of its own underlying data rather than assuming it."*

Use whichever paragraph honestly describes what you actually built by submission day — an accurate Part-A-only pitch beats an inflated Part-C pitch for a system that only has the T4 tables sitting empty.

D4. Closing checklist before you submit

- ☐ Every T1 and T2 table built, migrated, and seeded
- ☐ All 22 core queries (Part A §7) run and their output captured
- ☐ Negative tests, concurrency test, performance test, integration test all captured with real output (Part A §13)
- ☐ ALTER + DROP migration present and demonstrated
- ☐ README + individual git contribution history for both members
- ☐ ER diagram generated from the final DDL, not drawn before it

- ☐ Demo script rehearsed at least twice end-to-end (Part A §15)
- ☐ If any T4 objects were built: their negative tests and at least one full state→decision→intervention→outcome walkthrough are in the test evidence, and the report's claims about impact are phrased as "associated with," not "caused"